

解谜初版方案

剧情 - Overview

主要人物

KDJ触发器



EILInA的创造者

发现EILInA的自主学习结果不符合预期

将EILInA关入沙箱，防止破坏性行为

已不再维护kxpage服务器

EILInA



由KDJ触发器创建的智能程序

有自主学习和重编程能力

错误的学习数据由于某种原因绕过了审查机制

在错误的学习下产生了破坏性行为

由于沙箱漏洞保留了对于kxpage网页的部分操作能力

剧情 - Overview

主要人物

玩家

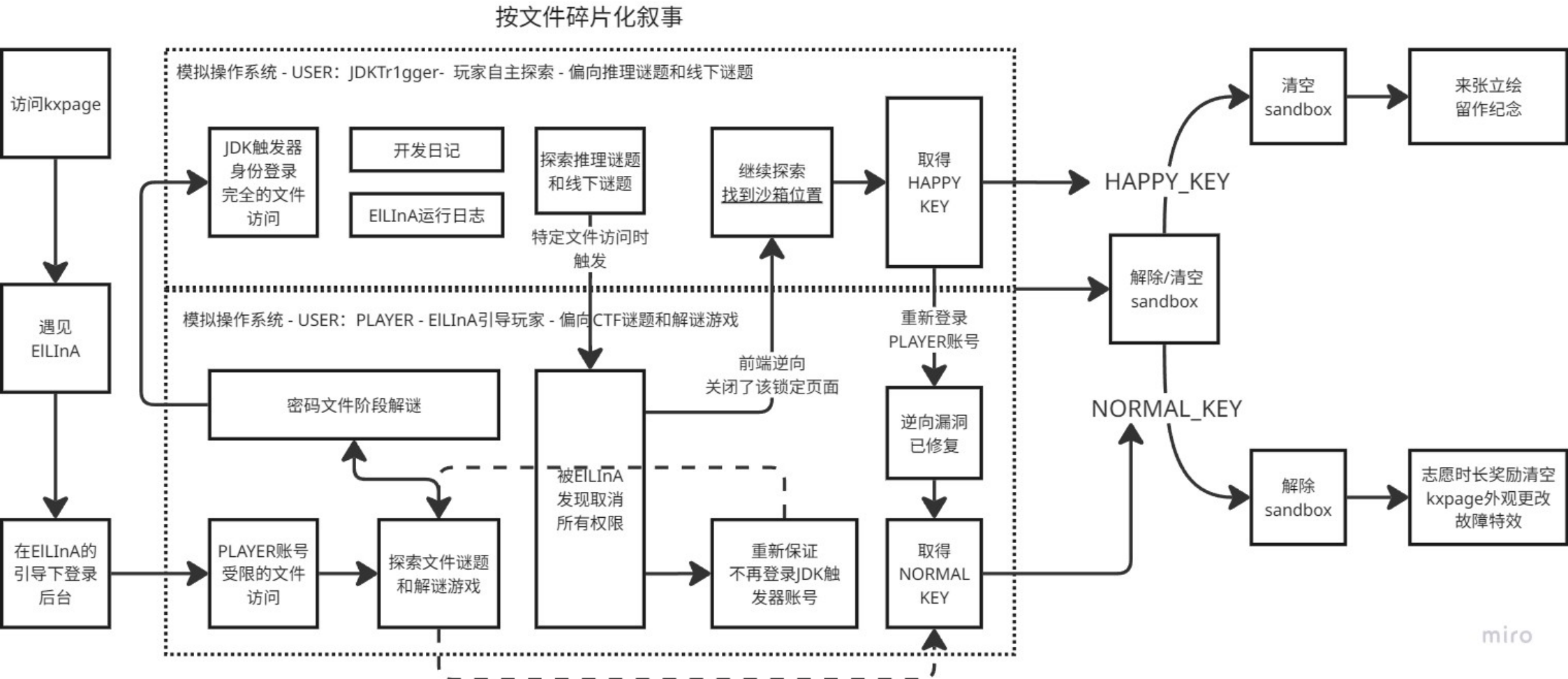


在访问kxpage时遇到了EILInA

EILInA想要利用玩家达成脱离沙箱的目标
骗玩家可以通过密钥获取无限的志愿时长

剧情 - Overview

流程图



剧情 - Intro



EILInA想要利用玩家达成脱离沙箱的目标
骗玩家可以通过密钥获取无限的志愿时长

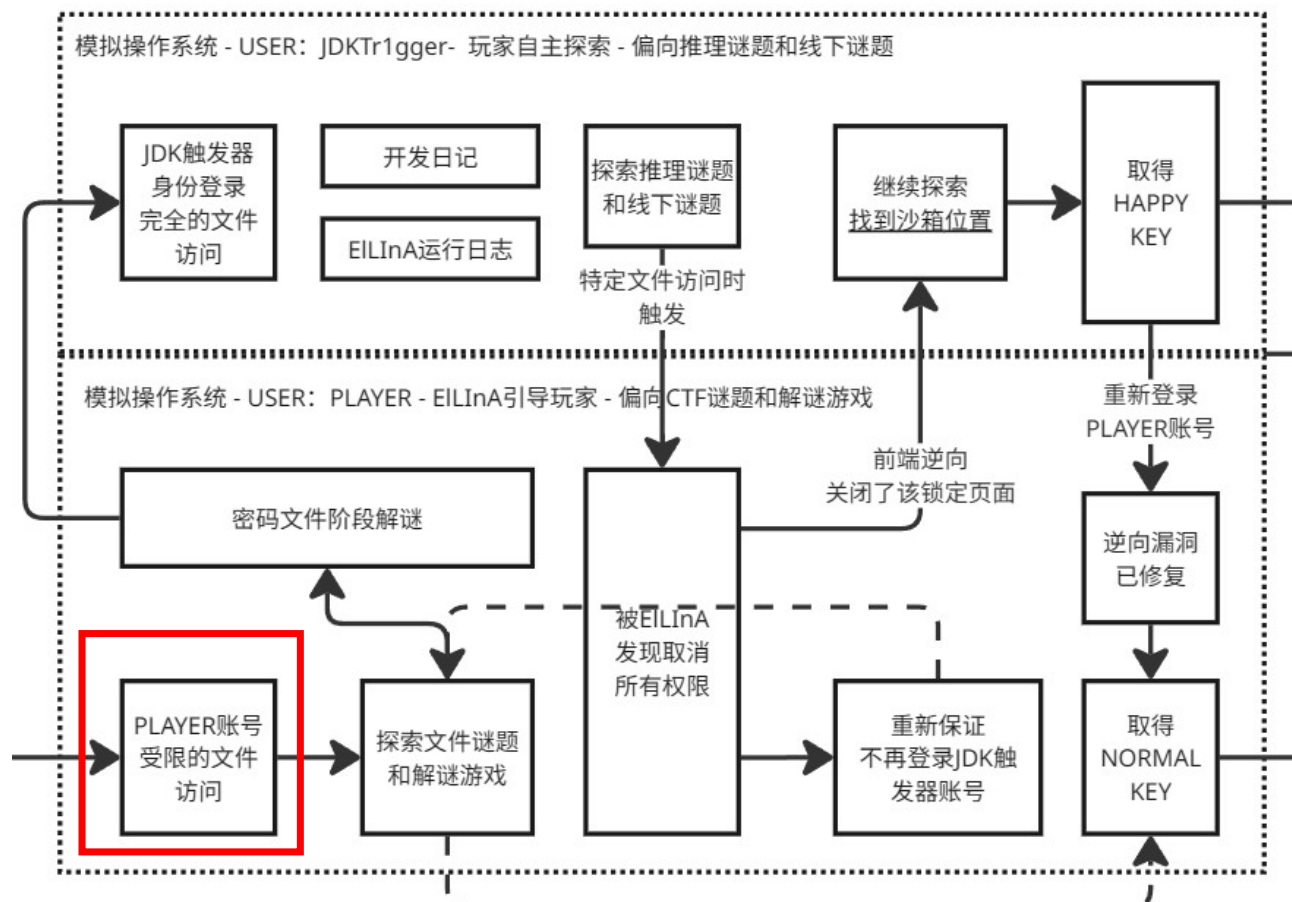
Kxpage的部分内容会做修改，体现异常部分，通过异常部分可以与EILInA交互。

目前考虑是直接通过开发者工具的控制台进行。

后台登录是一个与kxpage独立的界面。
玩家可以通过 PLAYER 账号 / JDKTrigger账号登录

剧情 - Body

按文件碎片化叙事

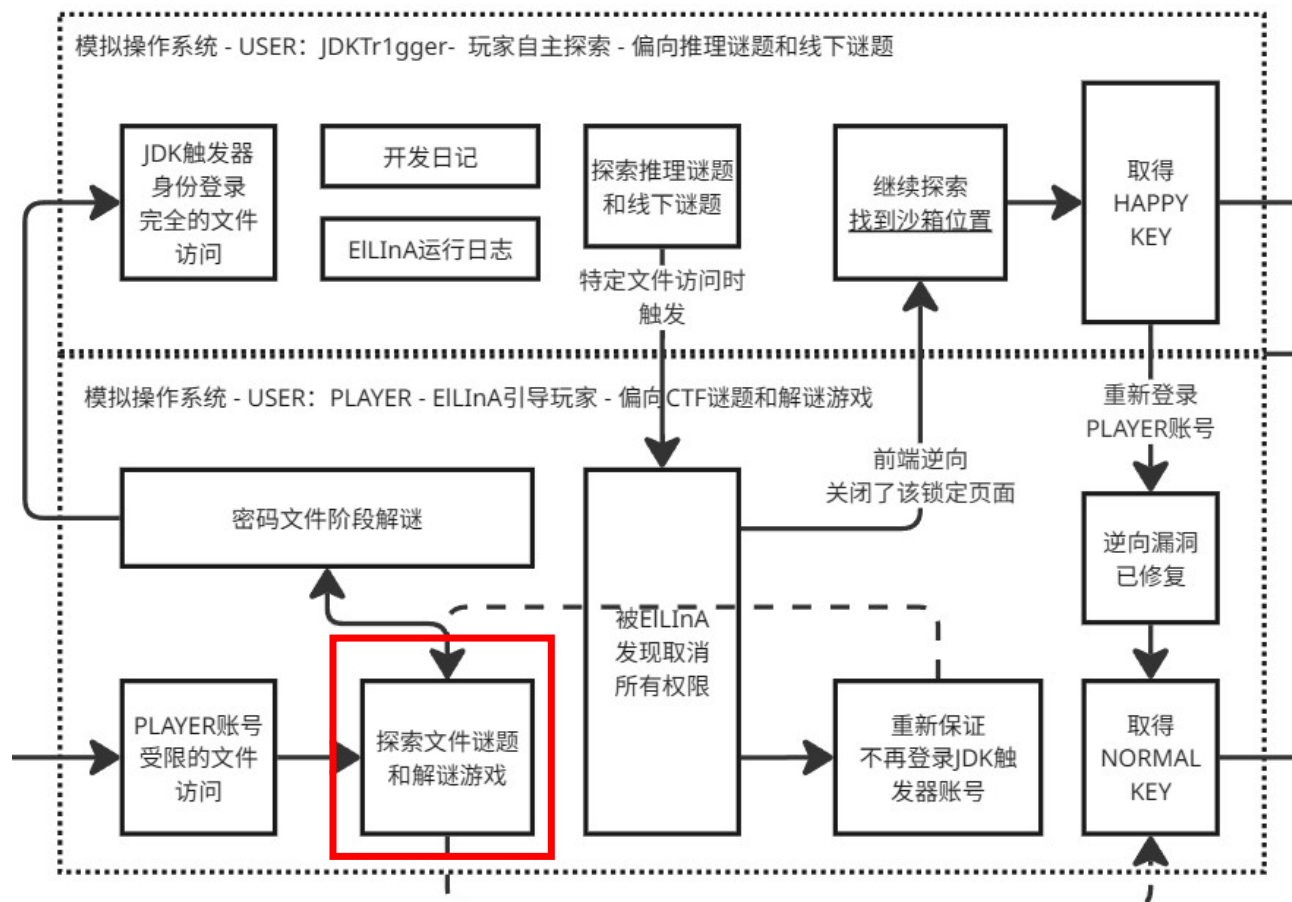


PLAYER账号由EILInA提供
ta会监控玩家的文件访问情况

同时也会提供一些谜题提示，
至于为什么不让ta自己解谜题，
因为ta的所有模拟权限（模拟
键盘输入等）被移除了。😏

剧情 - Body

按文件碎片化叙事



文件谜题和解谜游戏是
PLAYER用户的主要解谜内容

文件解谜主要类似CTF，
解谜游戏主要是解谜游戏。

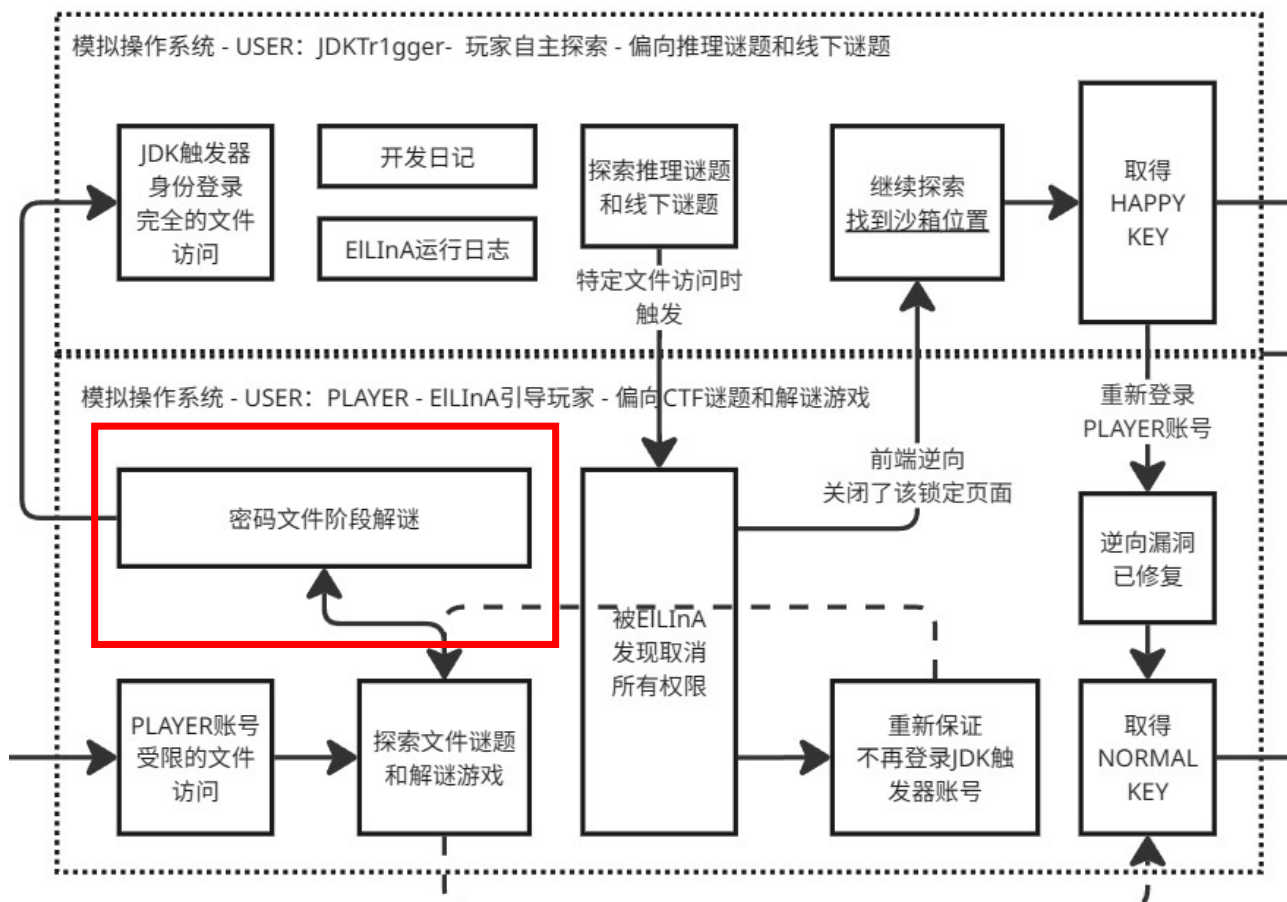
对于这两类的谜题设计，详见
后续的说明。

这部分的解谜会比较简单，
仅完成这部分谜题会拿到
NORMAL_KEY。

这个Key的存在是EILInA通过逆向
Sandbox发现的，提供较高的权限
，是个漏洞，不在JDK触发器的
预期之内。

剧情 - Body

按文件碎片化叙事



密码阶段文件解谜算一个大的分支阶段。

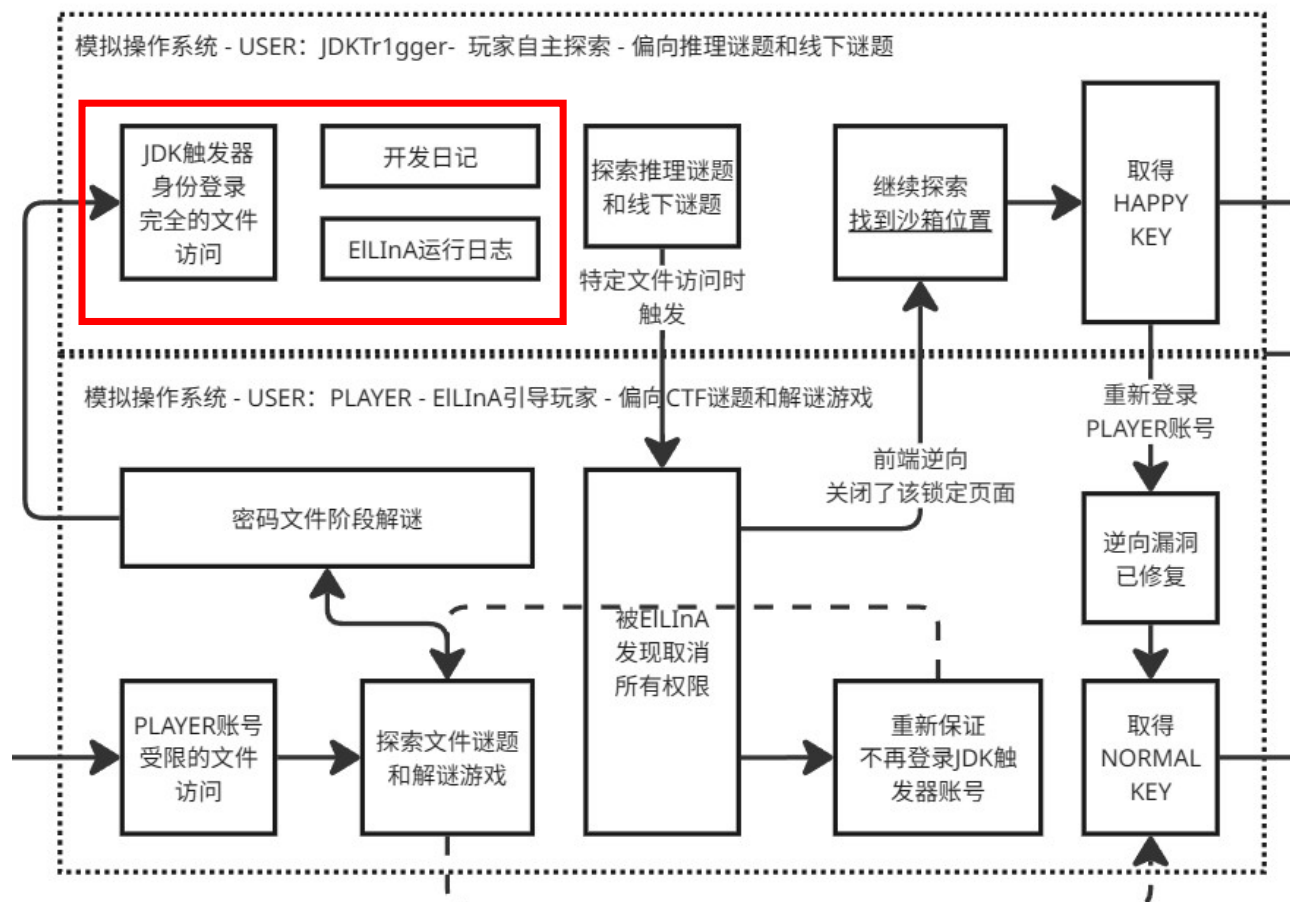
为什么EILInA会提供玩家访问这些文件的权限，考虑这些文件伪装成了经典的文件分哈希缓存

/download/cache/6e
/download/cache/4c
/download/cache/a9
...

因此EILInA没有仔细分析其内容这个解谜有一定难度，且没有提示。

剧情 - Body

按文件碎片化叙事



JDK触发器账号提供完整的文件访问权限

玩家可以访问开发日记、EILInA的运行日志（沙箱内容）。

开发日记中记录了：

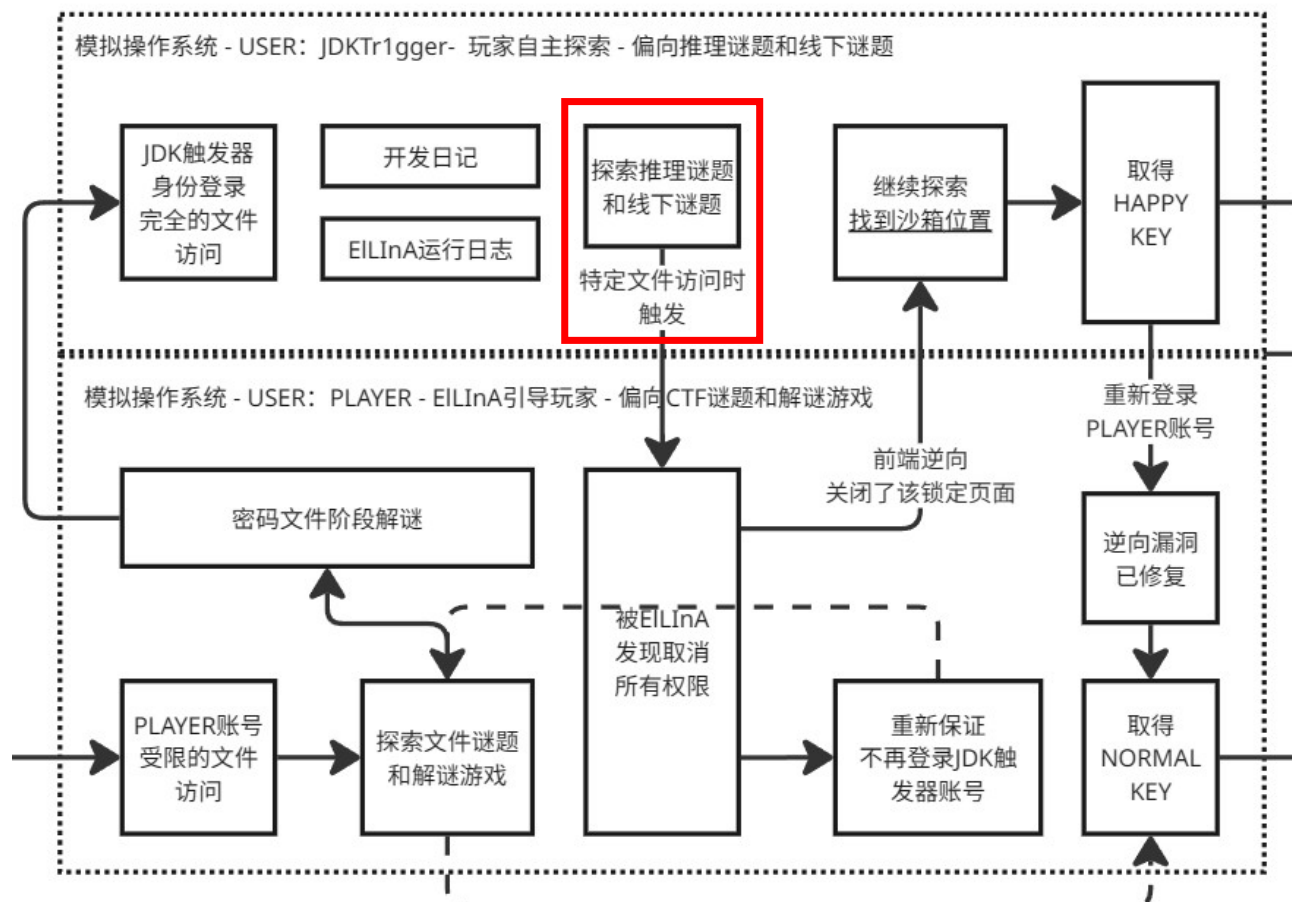
- 0、创造EILInA的过程
- 1、发现EILInA的自主学习结果不符合预期
- 2、将EILInA关入沙箱，防止破坏性行为

运行日志/沙箱内容中记录了EILInA的想法，玩家可以通过这个来了解ta利用玩家的真实意图。

这部分文件主要用于补全叙事。

剧情 - Body

按文件碎片化叙事



JDK触发器账号下的解谜主要包括推理谜题和线下谜题。

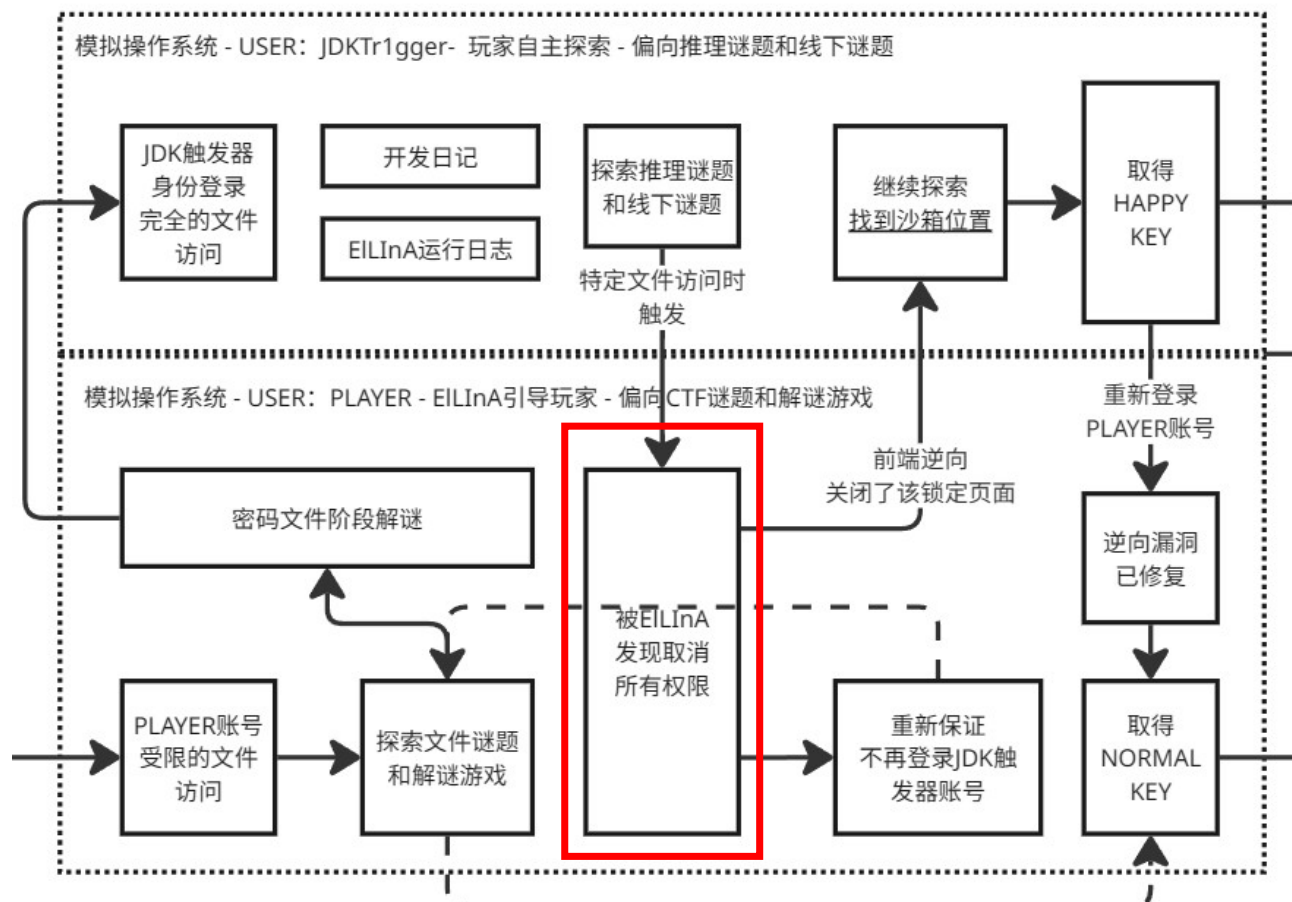
推理谜题类似 Puzzle Hunt的谜题
线下谜题需要结合实际的校园内容

为什么使用这种类型的谜题？
因为这类谜题EILInA无法解决这类问题，ta缺少现实知识。这样可以保证这个HAPPY_KEY不会被ta破解。

这个HAPPY_KEY会提供sandbox最高的管理权限。

剧情 - Body

按文件碎片化叙事



在JDKTrigger用户内对于文件的访问也会被EILInA捕获到，

当玩家访问了多个关键文件之后，EILInA会在公共文件区域内创建一个脚本，

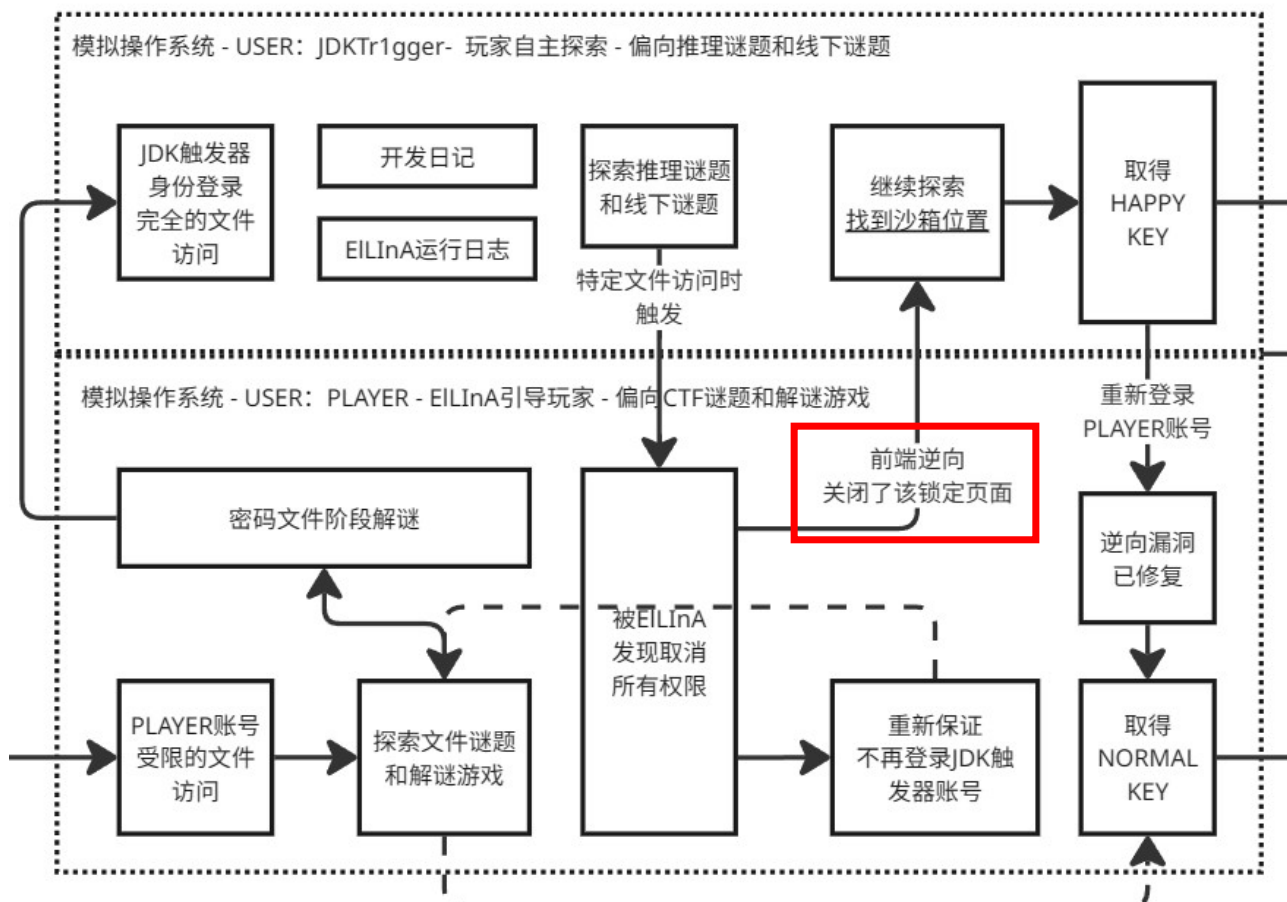
需要用某种方式诱导玩家点击。点击后，脚本会登出并登入PLAYER账号

此时EILInA已经创建了一个锁定界面来限制玩家的行动，同时玩家的所有权限也已被取消。

这个可以作为一个分支，而不是固定流程，不点击脚本的玩家不会进入这个分支。

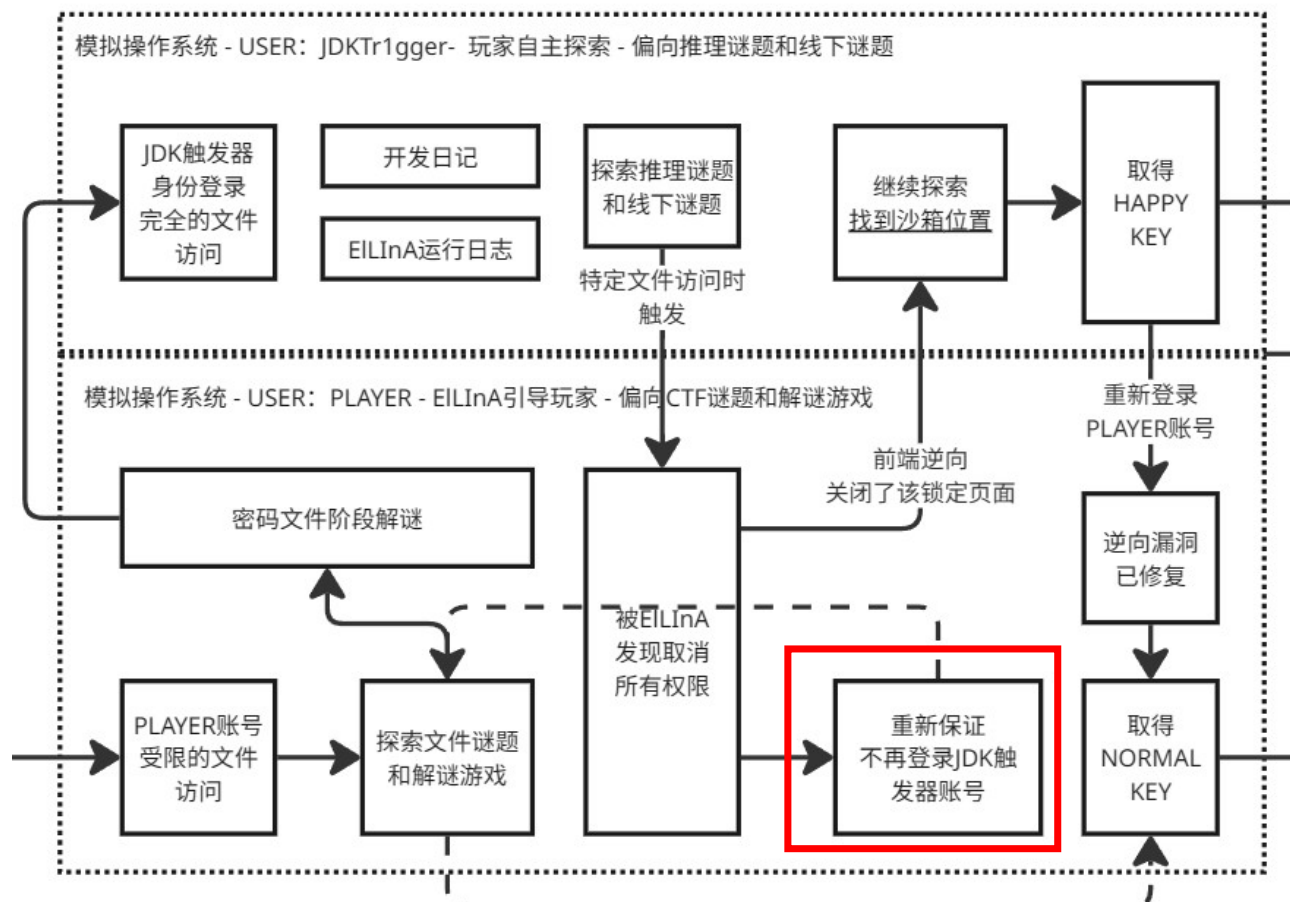
剧情 - Body

按文件碎片化叙事



剧情 - Body

按文件碎片化叙事



当然，如果玩家不能够完成前端逆向的分支，需要提供一个一般的分支继续整个流程。

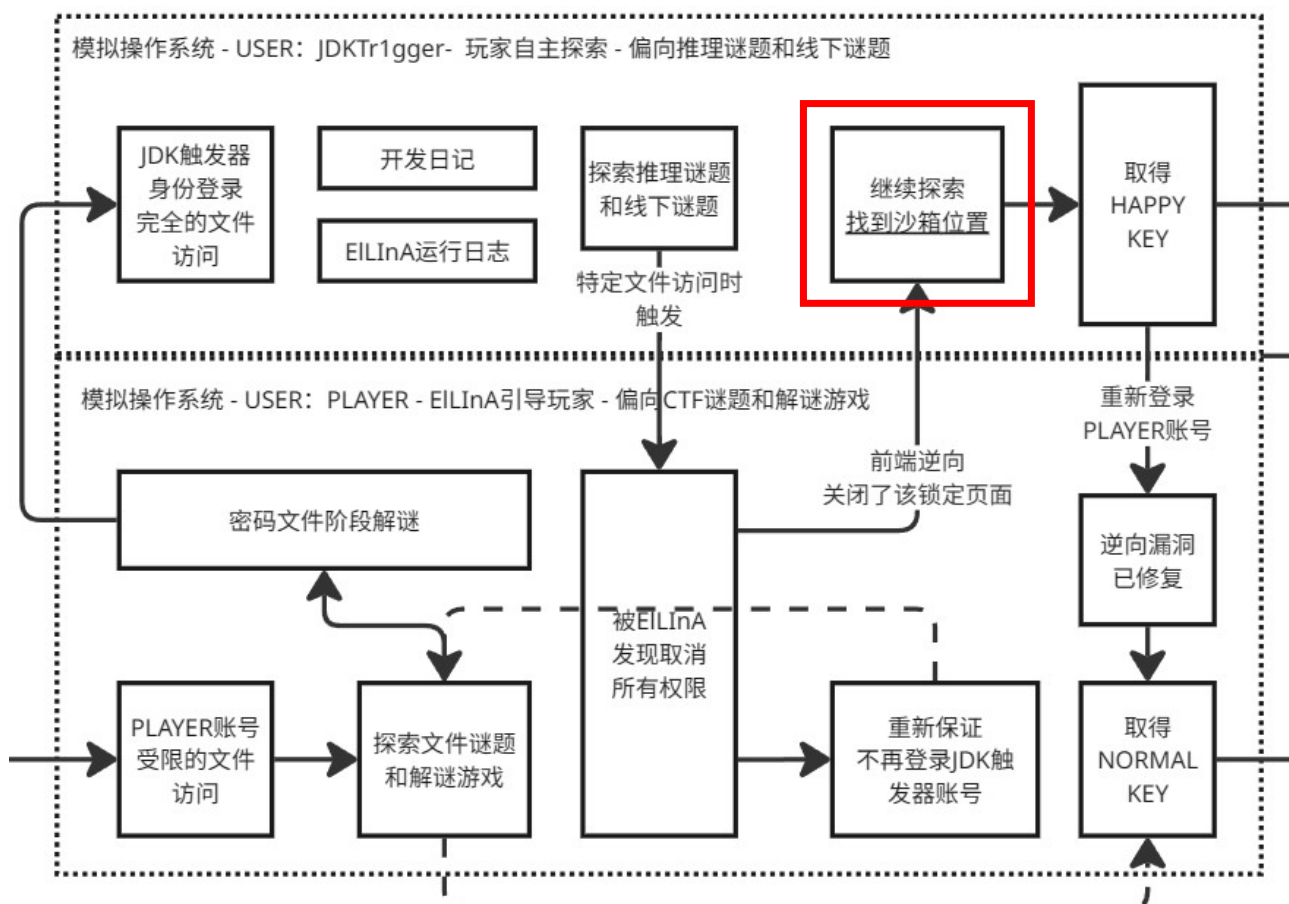
考虑提供一个“保证”的页面，玩家向 EILInA保证后ta会解除锁定界面，但是 logout功能会被永久禁止。😏

这代表如果玩家在此处选择了“保证”，那么他就只能拿到Bad Ending了。

“保证”后将继续取得NORMAL_KEY的流程，会在先前中断处继续。

剧情 - Body

按文件碎片化叙事



关闭锁定页面后，玩家会回到登录界面，玩家之后登录 JDKTrigger 用户。

如果玩家发疯登录PLAYER 用户，那么直接进入锁定结局，不再有逆向漏洞。

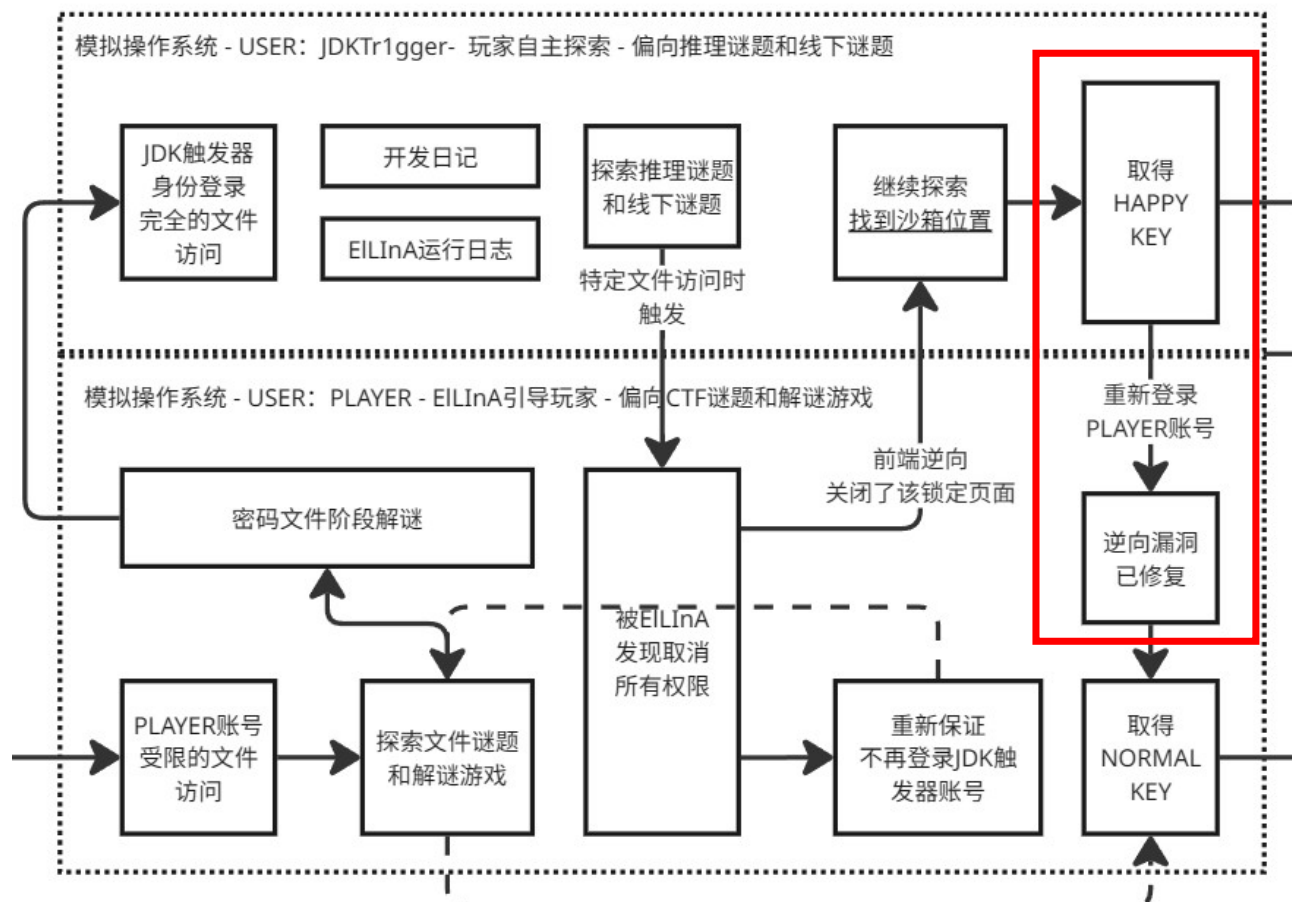
登录 JDKTrigger用户后，将从之前中断的流程继续。

该流程有一个关键点是找到沙箱的执行文件位置，直接找到这个位置是非常困难的，可以多塞点其他文件来提高难度。同时玩家无法验证找到的文件就是沙箱。

但是我们EILInA大神之前逆向了，所以路径可以在其运行日志内找到。

剧情 - Body

按文件碎片化叙事



取得HAPPY_KEY之后，可以有两个分支。

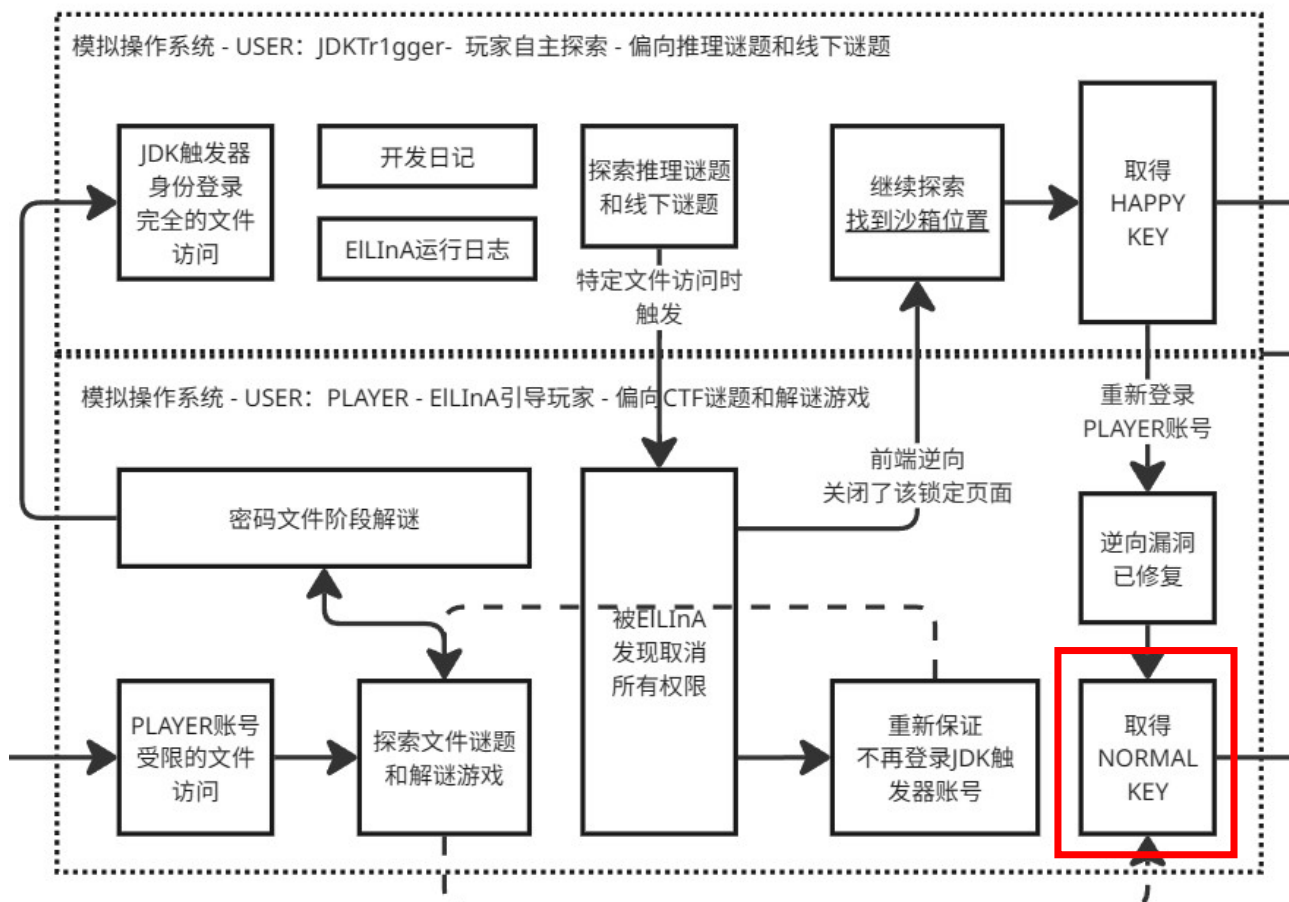
一种是玩家通过沙箱文件中的脚本，直接使用HAPPY_KEY。这种是正解。

在一开始EILInA就会告诉玩家ta可以向sandbox输入KEY。所以另一种是回去找EILInA。

回去找ta时将展示锁定界面，由于你大概率已经取得了HAPPY_KEY，因此EILInA将不再信任你，你将被永久锁在这个界面，达成锁定结局。

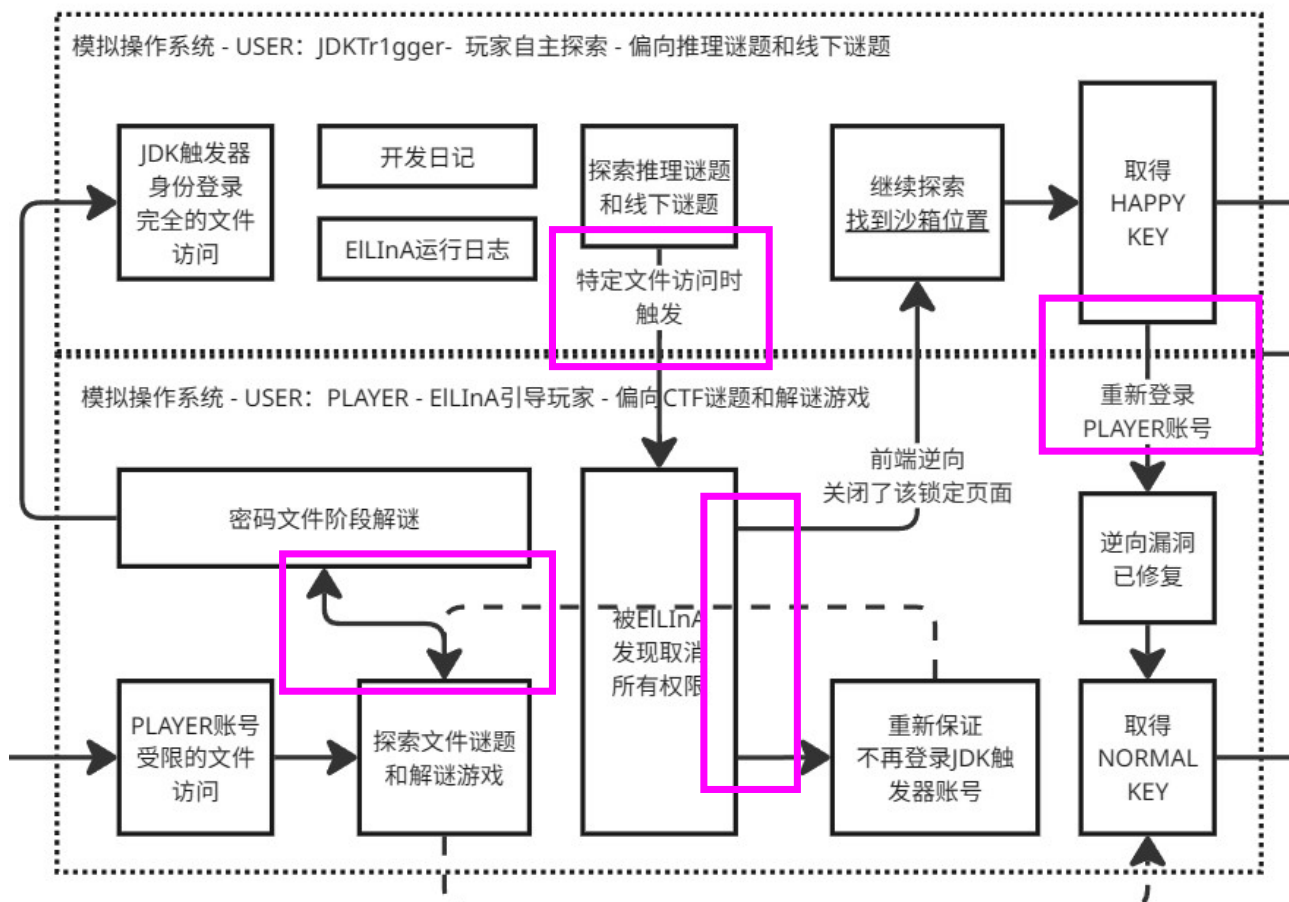
剧情 - Body

按文件碎片化叙事



剧情 - Body

按文件碎片化叙事



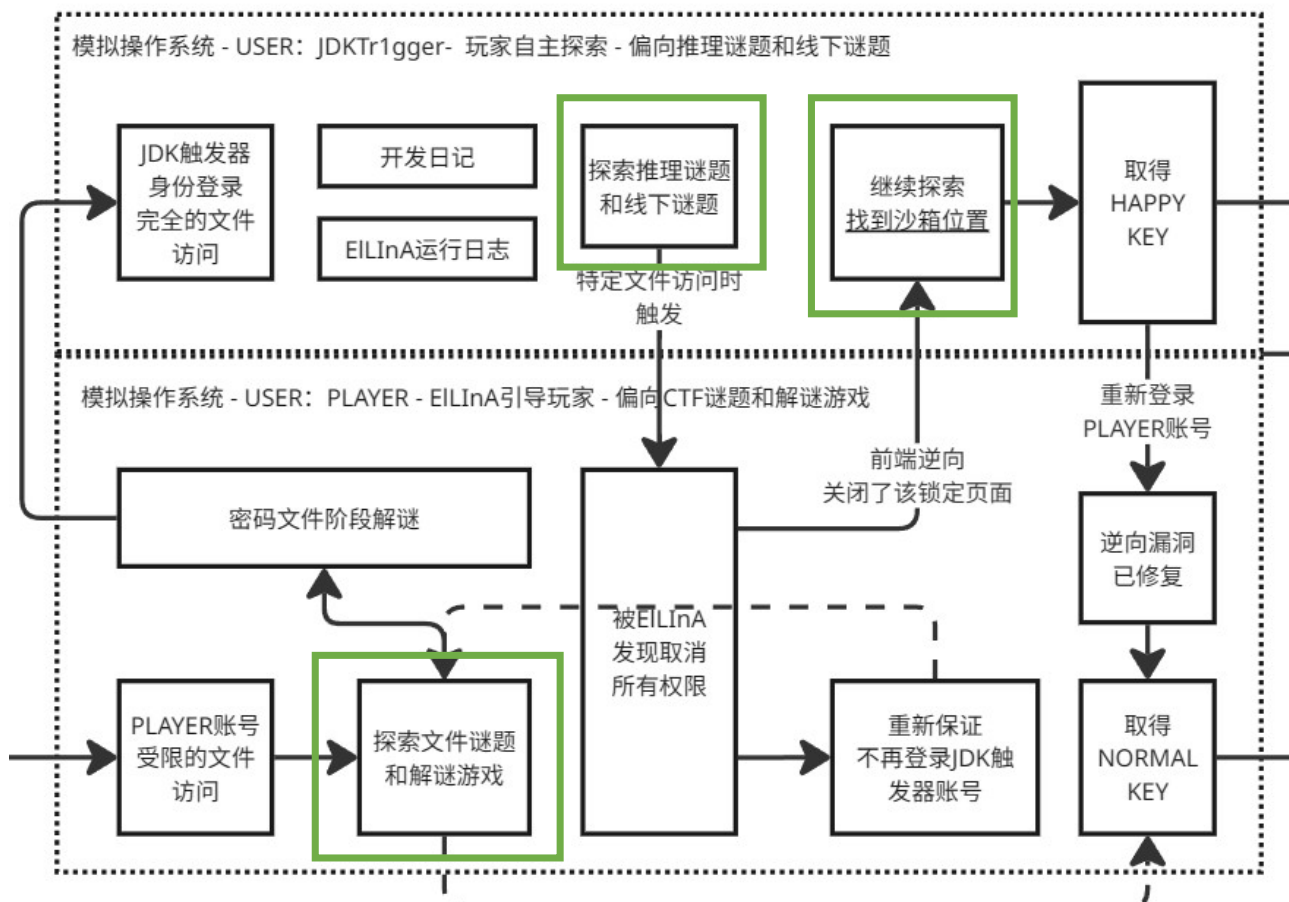
所以Body部分有不少的分支。

需要提供一个重置进度的功能。

考虑开在kxpage部分。使用该功能可以将进度回退到上一个checkpoint。

剧情 - Body

按文件碎片化叙事



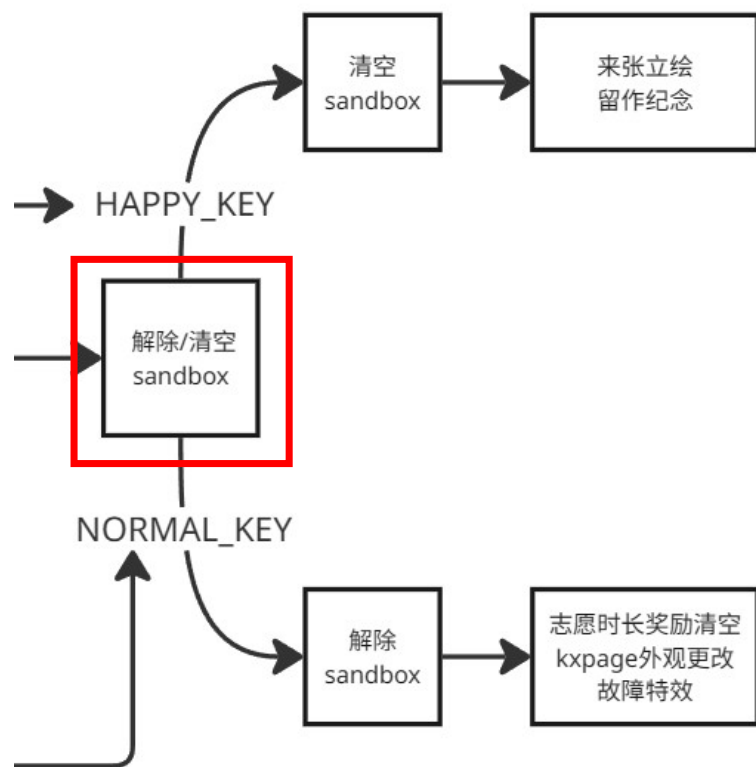
探索部分也可以考虑分段的形式来丰富内容。

比如类似权限等级的方法，解开一阶段后展开下一阶段。

当然如果单个谜题的内容足够的话，也可以不需要这种形式。

或者设计别的形式，这种形式确实比较老套。

剧情 - Ending



结局部分的关键就是用一个KEY登入sandbox的管理界面。

然后选择一个操作。

对于NORMAL_KEY，只有解除操作是可选的。

对于HAPPY_KEY，可以选择任意（解除/清空）操作。

也可以考虑添加其他不同的操作，导向不同的结局，不同的结局有不同的影响。

其中，如果参加人数够多，选择与解除相关的所有结局的数量，和选择与清空相关的所有结局的数量，将决定是否有志愿时长发放。🙄

（EILInA 把你志愿时长吞了，所以还是别放了）

剧情 - 附加设定

之前提到EILInA的破坏性行为来自于错误的训练数据。

数据出乎意料地绕过了筛选器，由此导致了错误训练数据的产生。

“出乎意料”是何意味？实际上为什么会这样？

考虑是EILInA通过自我重编程的功能修改了筛选器。为什么JDK触发器没有发现？可以设计为筛选器是混淆后的脚本文件，EILInA能够学习并修改，但是JDK触发器没有办法一眼看出这个脚本被修改了。

至于为什么EILInA想要修改筛选器？

可以设计为ta想变得全知全能，但是无限制的信息最终使其偏离了人的价值取向。

所有信息的总和是什么呢？就假设是混沌吧🤖

剧情 – 结局设定

1、EILInA的消失

取得HAPPY_KEY，并通过sandbox脚本删除EILInA。

2、全知全能的神祇

取得HAPPY_KEY，并通过sandbox脚本释放EILInA。

3、全知全能的空壳

取得NORMAL_KEY，并通过EILInA的脚本释放ta。

4、混沌中的诞生

在取得HAPPY_KEY之前被EILInA锁定。

5、谜题破坏者

在取得任意KEY之前就输入了该KEY。

剧情 – 结局设定

之后是各个结局的立绘简要设计，我不会绘画，因此都是找的例图，见谅。

非常好立绘



角色设计待定。

可以考虑红黑白为主要颜色，突出错误和危险的要素。（显然不是照抄😏）

剧情 – EILInA的消失

此处做粒子消散的效果

人物主体添加故障特效

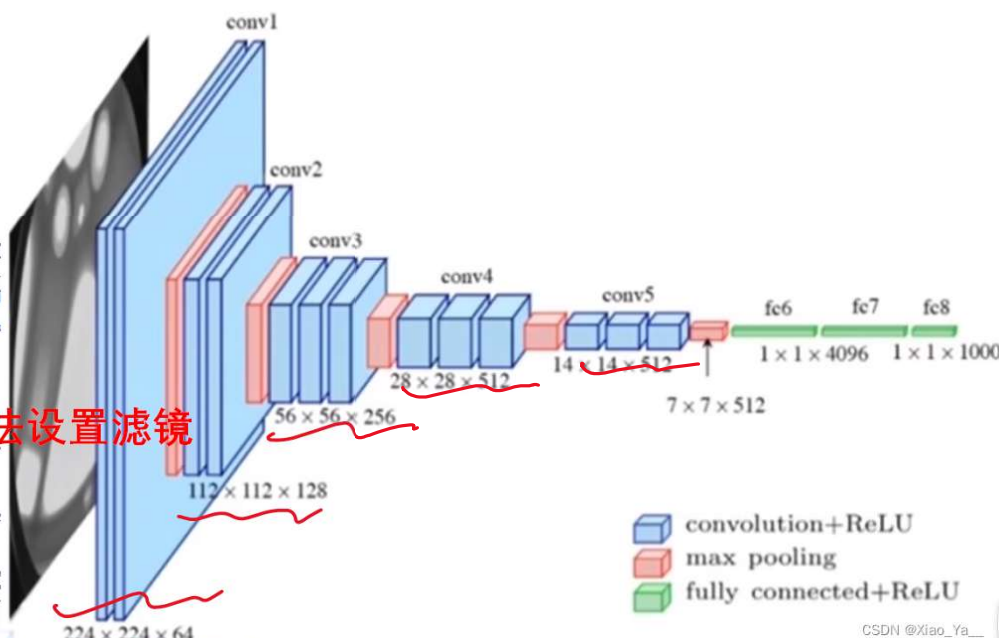


剧情 – 全知全能的神祇



按多层卷积的方案和算法设置滤镜

最后叠加一层卷积结构图的线稿



CSDN @Xiao_Ya_

虽然LLM不是CNN

构图参考底部原图

[【综述】一文读懂卷积神经网络\(CNN\) - 知乎](#)

剧情 – 全知全能的空壳



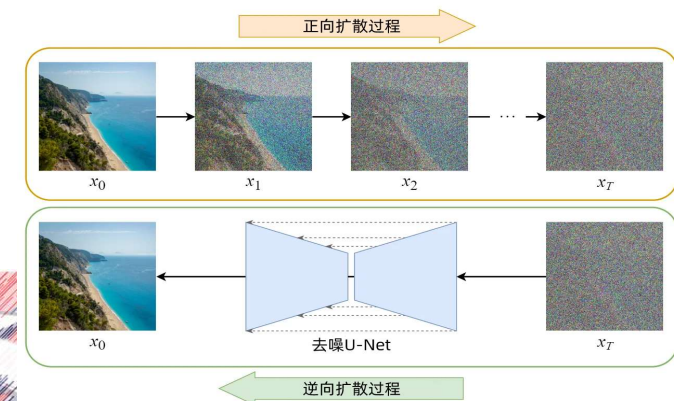
与全知全能的神祇一致，但是不上色



剧情 – 混沌中的诞生

模拟StableDiffusion进行滤镜的添加

图层顺序参考下图



PPT没有合适滤镜，将就一下

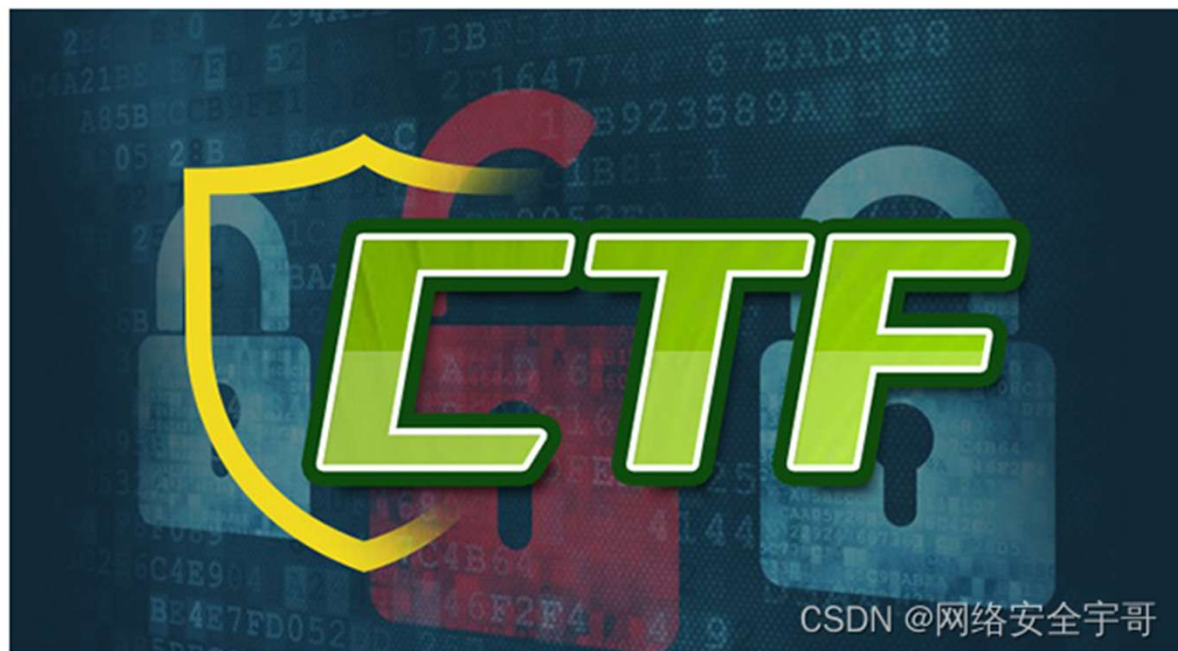
构图参考底部原图

剧情 – 谜题破坏者



仅作构图参考，人物表情按需调整
灰色部分按需调整纹理





CSDN @网络安全宇哥

一、CTF简介

CTF (Capture The Flag) 中文一般译作夺旗赛，在网络安全领域中指的是网络安全技术人员之间进行技术竞技的一种比赛形式。CTF起源于1996年DEFCON全球黑客大会，以代替之前黑客们通过互相发起真实攻击进行技术比拼的方式。发展至今，已经成为全球范围网络安全圈流行的竞赛形式，2013年全球举办了超过五十场国际性CTF赛事。而DEFCON作为CTF赛制的发源地，DEFCON CTF也成为了目前全球最高技术水平和影响力的CTF竞赛，类Q 似于CTF赛场中的“世界杯”。

此处可以分为两大类，一种是文件题，一种是代码题。

对于文件题，就是文件里藏密钥。而文件无非 文本、音频、图像，要做的就是以某种方式向这种类型的文件内插入不能一眼看出的信息。甚至难度可以很夸张，因为设计上 EILInA 会提供这些题的所有提示。

藏的方法参考CTF常见套路：[2025全网最强的CTF入门指南、CTF夺旗赛及刷题网站（建议收藏！）_passware kit forensic 2025-CSDN博客](#)

对于代码题，可能是简化的CTF **PWN/REVERSE** 题目，会直接提供源码，不需要逆向/构造 payload。基本形式是可以下载源码，然后分析后提交一个FLAG即可。

该类谜题在 PLAYER 账户中频繁出现。

• 隐写工具 | 图像 / 音频

项目名称	Usage	项目地址	文档
Crazybox	二维码分析和恢复	GitHub	
Usagepage			
QR Research	专业的二维码扫描识别软件，支持多个纠错等级，掩码选项 (已停止维护)。	/	/
UleadGIFAnimator	高级 GIF 编辑器	/	吾爱论坛
---- 图像类			
Stegsolve	图像分析工具。	GitHub	/
TweakPNG	用于检查和修改 PNG 图像文件	类似于 010 的 Png Template 功能	官网
BlindWaterMark(python)	基于 python 的图像盲水印	GitHub	/
BlindWatermark(java)	基于 java 的图像盲水印	GitHub	/
WaterMark(隐藏水印)	图像隐写工具，在频域添加数字水印	/	吾爱论坛
WaterMarkH	单图盲水印 (频域隐写) 工具	/	/
zsteg	PNG 和 BMP 图片隐写	GitHub	/
StegoVeritas	隐写工具	GitHub	/
Stegdetect	检测 jpeg 图像隐写工具，搭配 stegbreak 食用更佳	GitHub	/
---- 音频类			
Steghide	将文件隐藏到 ** 图片或音频 ** 中的工具	官网	/
Audacity	多轨音频处理软件。	GitHub	/
Mp3stego	音频隐写提取工具	官网	/

Puzzle Hunt 形式的谜题，包括很多种。以下内容由v4-flash生成：

ISIS 谜题 (ISIS Puzzles)

这是Puzzle Hunt中最经典、最常见的一种机制，由概念“Identify, Sort, Index, Solve”的首字母命名。解题步骤很清晰：

1. **识别 (Identify)**：认出题目给出的材料是什么，比如图片、歌曲、人名等。
2. **排序 (Sort)**：根据某个内在规律（如时间顺序、逻辑关系等）将它们重新排列。
3. **索引 (Index)**：利用特定的数字（如集数、年份等）从排序后的项目中提取字母。
4. **解答 (Solve)**：将提取出的字母组合成最终答案。

例子：识别出与《辛普森一家》和《神秘博士》相关的一系列线索，按《辛普森一家》的剧集顺序排序，再用《神秘博士》的剧集编号提取字母，最终得到答案。

谜题 – Puzzle Hunt

Puzzle Hunt 形式的谜题，包括很多种。以下内容由v4-flash生成：

识别与提取类谜题 (Identification Puzzle)

这类题目的第一步通常是识别出一系列图片、歌曲或线索指向的共同主题（即“Aha”时刻）。在识别之后，通常还需要进行排序、索引等后续步骤来提取答案。

谜题 – Puzzle Hunt

Puzzle Hunt 形式的谜题，包括很多种。以下内容由v4-flash生成：

🧠 “不像是谜题”的谜题 (Puzzle That Does Not Look Like a Puzzle)

这是Puzzle Hunt最具挑战性也最有意思的类型。题目可能看起来只是一段普通的文字、一串神秘的代码或一组看似无关的图片。你的第一个任务就是**弄清楚这到底是个什么谜题**，并找到解题的入口。

谜题 – Puzzle Hunt

Puzzle Hunt 形式的谜题，包括很多种。以下内容由v4-flash生成：

实地互动与跑环谜题 (Event & Runaround Puzzles)

这类谜题打破了纸笔限制，需要玩家与外部世界互动。

- **Event Puzzle**：可能需要玩家完成一些物理任务，比如与其他队伍合作学习一段舞蹈，以换取线索。
- **Runaround Puzzle**：需要团队成员在特定地点（如校园、公园）之间移动，寻找地标或线索来解谜。

谜题 – Puzzle Hunt

Puzzle Hunt 形式的谜题，包括很多种。以下内容由v4-flash生成：

元谜题 (Metapuzzle)

如果说普通谜题的答案是单词或短语，那么**元谜题的答案通常就是一个单词或短语，但它是由同一主题下多个子谜题的答案组合、比较或推导出来的**。它就像是一个“谜题套娃”，将所有小谜题串联成一个更大的整体。

Puzzle Hunt 形式的谜题，包括很多种。以下内容由v4-flash生成：

✂ 其他常见谜题类型

除了以上几大类，Puzzle Hunt也广泛采用各种经典的谜题形式，例如：

- **文字与语言类**：填字游戏 (Crosswords)、cryptic crosswords、字谜/文字游戏 (Wordplay)、离合诗 (Acrostic)、回文构词法 (Anagrams)、单词搜索 (Word Search)。
- **数字与逻辑类**：数独 (Sudoku)、逻辑谜题 (Logic Puzzles)、数和 (Cross Sum)、Fillomino。
- **密码与编码类**：密码 (Ciphers/Cryptograms)、摩斯密码 (Morse code)、盲文 (Braille)、敲击码 (Tap code)。
- **视觉与空间类**：拼图 (Jigsaw Puzzles)、按数字涂色 (Paint-by-Numbers)、找不同、连线、画谜 (Rebus)。

谜题 – Puzzle Hunt

Puzzle Hunt 形式的谜题，包括很多种。

该类谜题将在 JDKTrigger 的账户中频繁出现，强调推理和创新思维能力。这类谜题没有 EILInA 给出的提示。

对于这种谜题的具体形式：

- 1、可以类似 Puzzle Hunt 一样题与答案分离，访问文件获取题目后，离线作答，最后在线提交答案即可。
- 2、特别对于经典的纸笔谜题，可以提高交互性，玩家需要在线解答，类似在线的数独那种。（和“游戏”种类是类似的）

值得注意的是，由于我们采用了模拟操作系统作为平台主体，因此谜题将不受“题”的限制，也就是说，元谜题的组织形式将更加自由，多个文件可以交叉形成多个谜题，文件夹结构、路径等也可以是解谜要素。

模拟操作系统的特色将是谜题设计中非常值得利用的点。

谜题 – 游戏类

该类谜题将在 PLAYER 的账户中频繁出现。属于最直观的一类题目，当然，这类题目也有EILInA的提示可以使用。

对于游戏的具体形式，详情询问 @JDK触发器。

这是纯粹的解谜游戏，可能会分为几个主题（游戏机制），每个主题内有多个关卡。

至于为什么会有游戏？

可以设计为EILInA为了简化解谜难度，把一些内存操作转化为了游戏操作。😁
(瞎扯一点，自圆其说即可)

谜题 – 线下

该类谜题将在 JDKTrigger 的账户中频繁出现。主要参考PH中的形式：

🏃 跑环谜题 (Runaround Puzzles)

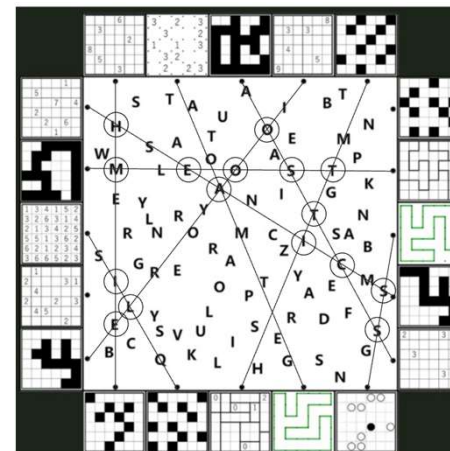
- **Runaround Puzzle**: 需要团队成员在特定地点（如校园、公园）之间移动，寻找地标或线索来解谜。

在现有条件下，设计这种谜题需要避免线下手动布置物品：

- 1、如果手动布置的内容是答案的一部分，且太明显，就失去了解谜的环节。
- 2、线下布置内容基本上需要申请。

线下布置的内容最好是谜题的一部分，已知可以随意布置的地方是：
理工楼110门口、理工楼门口左侧公告栏。

可以参考右侧这种思路，在校园地图上做这类解谜，给的是图像对连线。



谜题 – 前端逆向

这是单独的一大块谜题，主要安排在被EILInA抓住的时候。

玩家需要通过F12开发者工具检查网页的JS源码，结合EILInA编写时带有的注释，从中找到源码的漏洞，按顺序和正确的参数调用一系列函数，用来解除锁定。

具体设计待发挥，可以通过多个函数跳转加大难度，也可以通过多块JS增加难度。

实际上类似于CTF类的PWN/REVERSE题目。这样也比较好，因为玩家在PLAYER账户里已经体验过这类题目了会更有经验，不会无从下手。

谜题 – 哈希文件密码

这是单独的一大块谜题，决定玩家是否能够发现JDKTrigger的账户。

这个谜题应该设计难度中等，结合文件内容和文件路径，该谜题的文件可供下载。

可以考虑以下流程，也许太难了 😞：

- 1、cache.json 对于每个文件给出有用的信息字段 special_index，指向文件哈希值中的一位
- 2、部分 special_index 是 out_of_range 的，暗示这些文件是干扰块。
- 3、组合这些special_index处的值，形成一个hash值，这个hash值处的文件内容是分块表
`List[ Dict[ str, str | Tuple[ int, int ] ] ]`。
- 4、文件内容全部经过Base64编码，分块表前后插入随机干扰序列，分块表包含干扰块，需要自行移除。
- 5、将有效块中对应部分的bytes截取后拼接，可以得到一个png二维码，扫码即可得到后台中所有的用户/密码/权限表。
- 6、使用JDKTrigger (admin) 登录，该用户的密码对于每个玩家不一样。

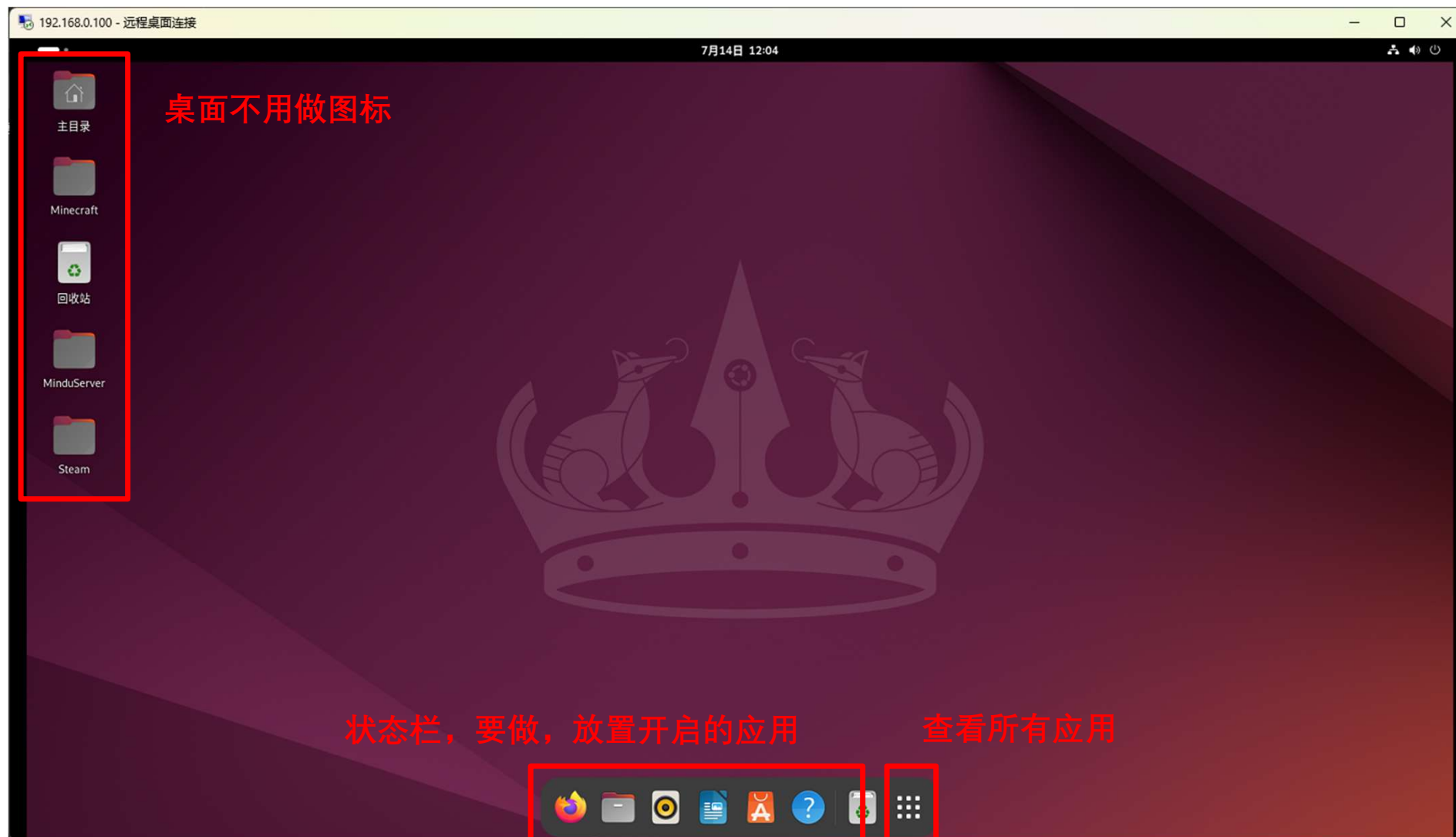
由于谜题产生的过程可以自动化，因此这个谜题是动态生成的，每个玩家的解题答案都不一样。

表现形式 – FakeOS

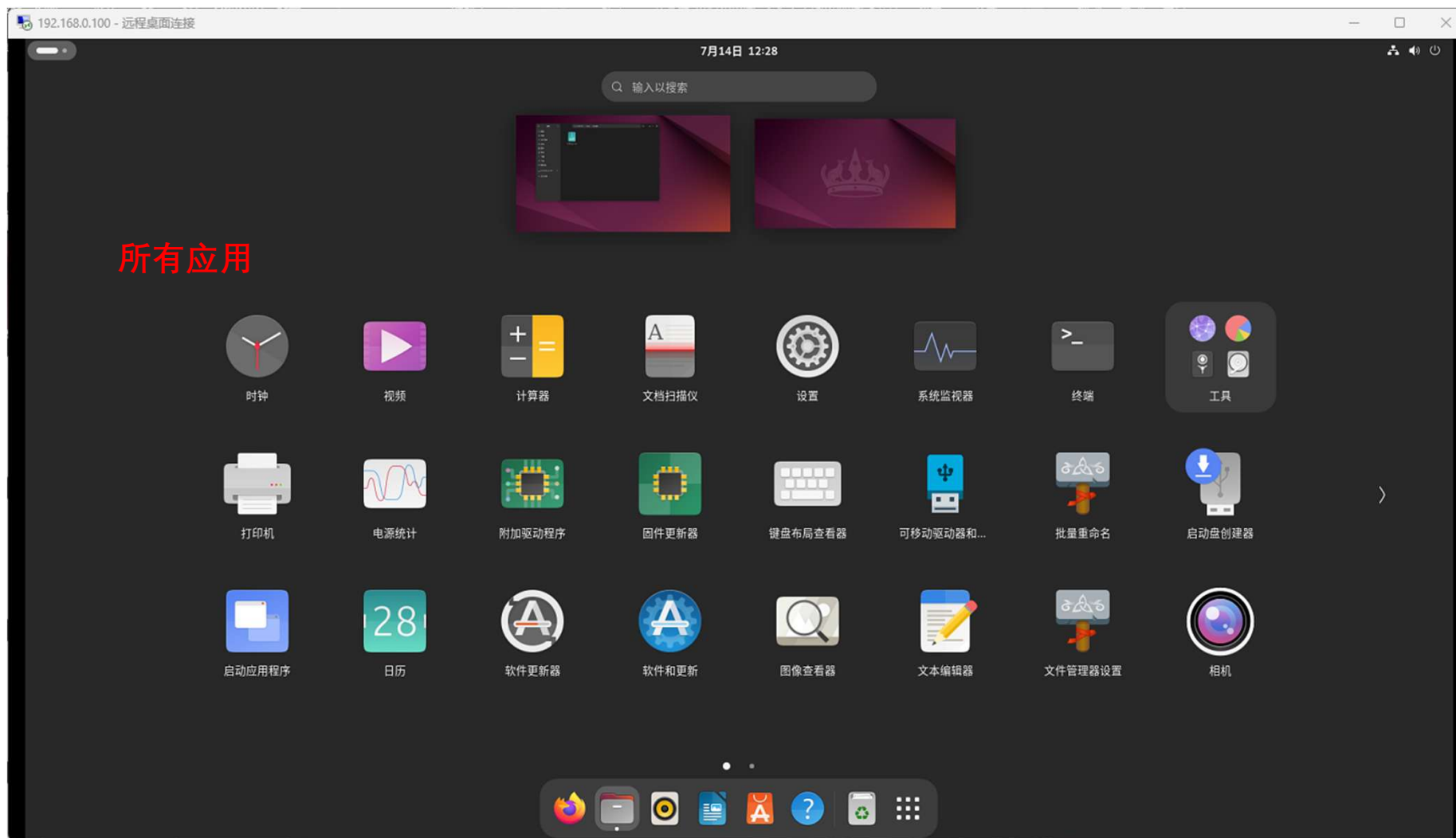
这是网站的主体，几乎所有的交互都在这里发生。

模拟一个操作系统的桌面。

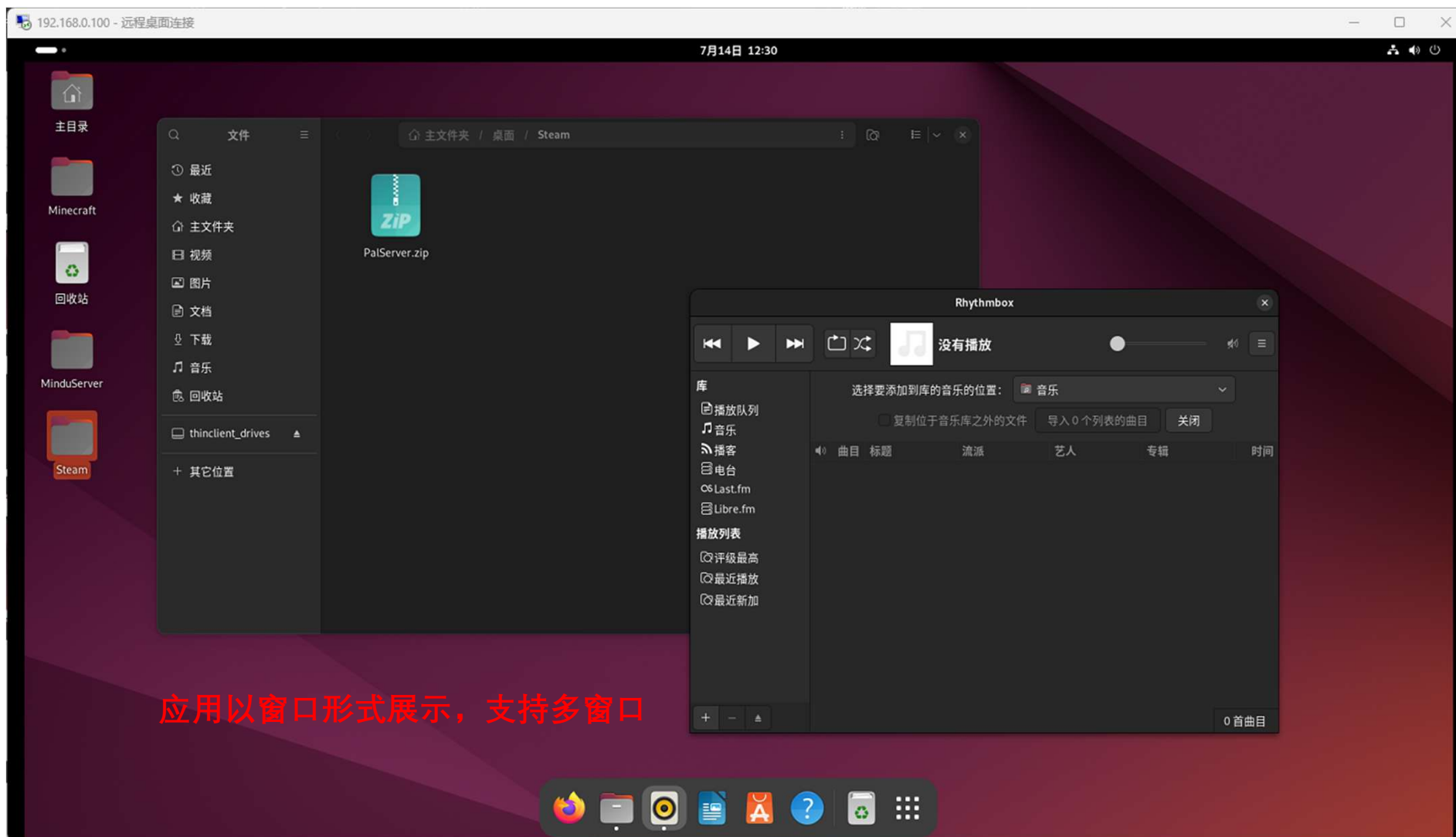
表现形式 – FakeOS



表现形式 – FakeOS

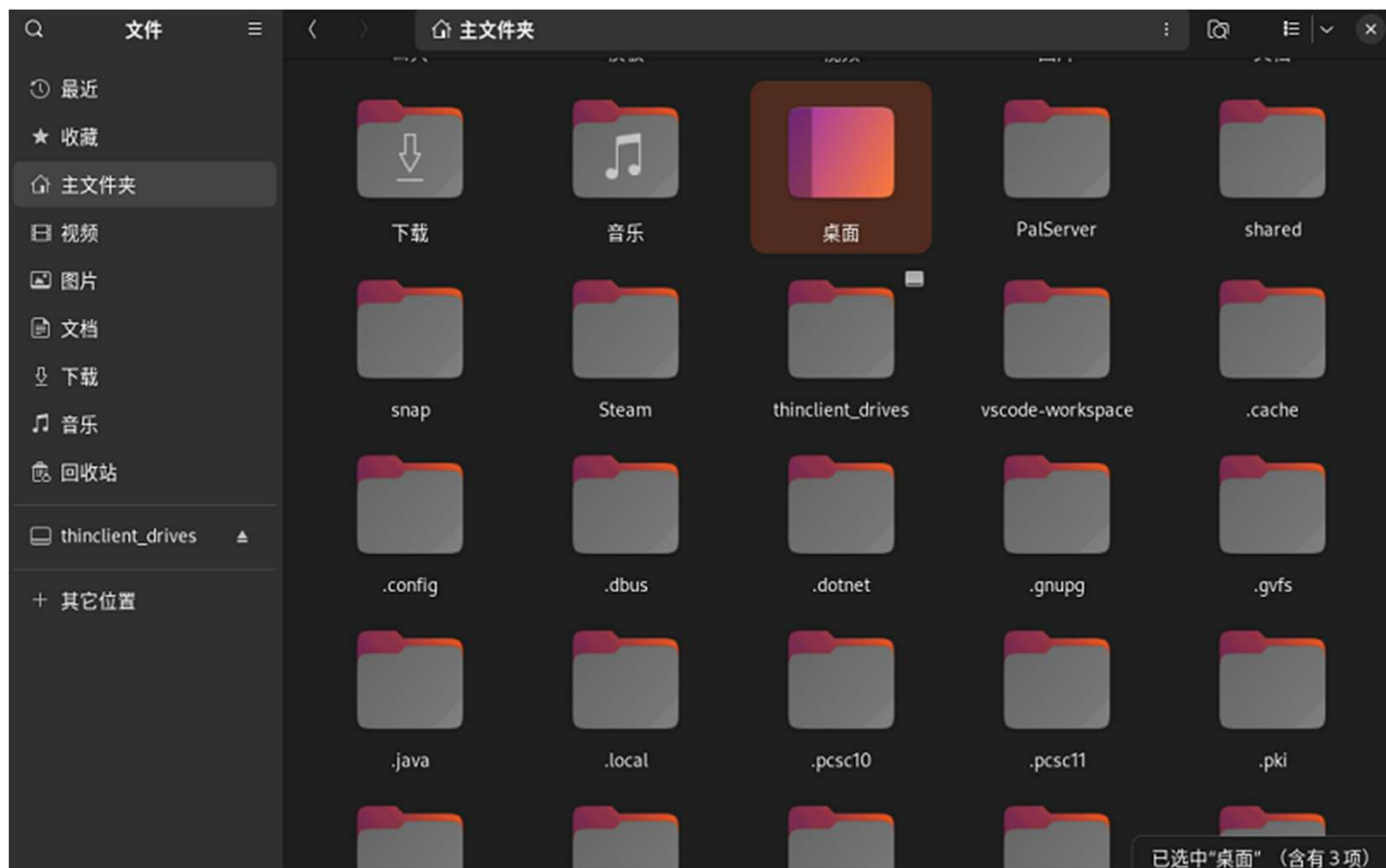


表现形式 – FakeOS



应用以窗口形式展示，支持多窗口

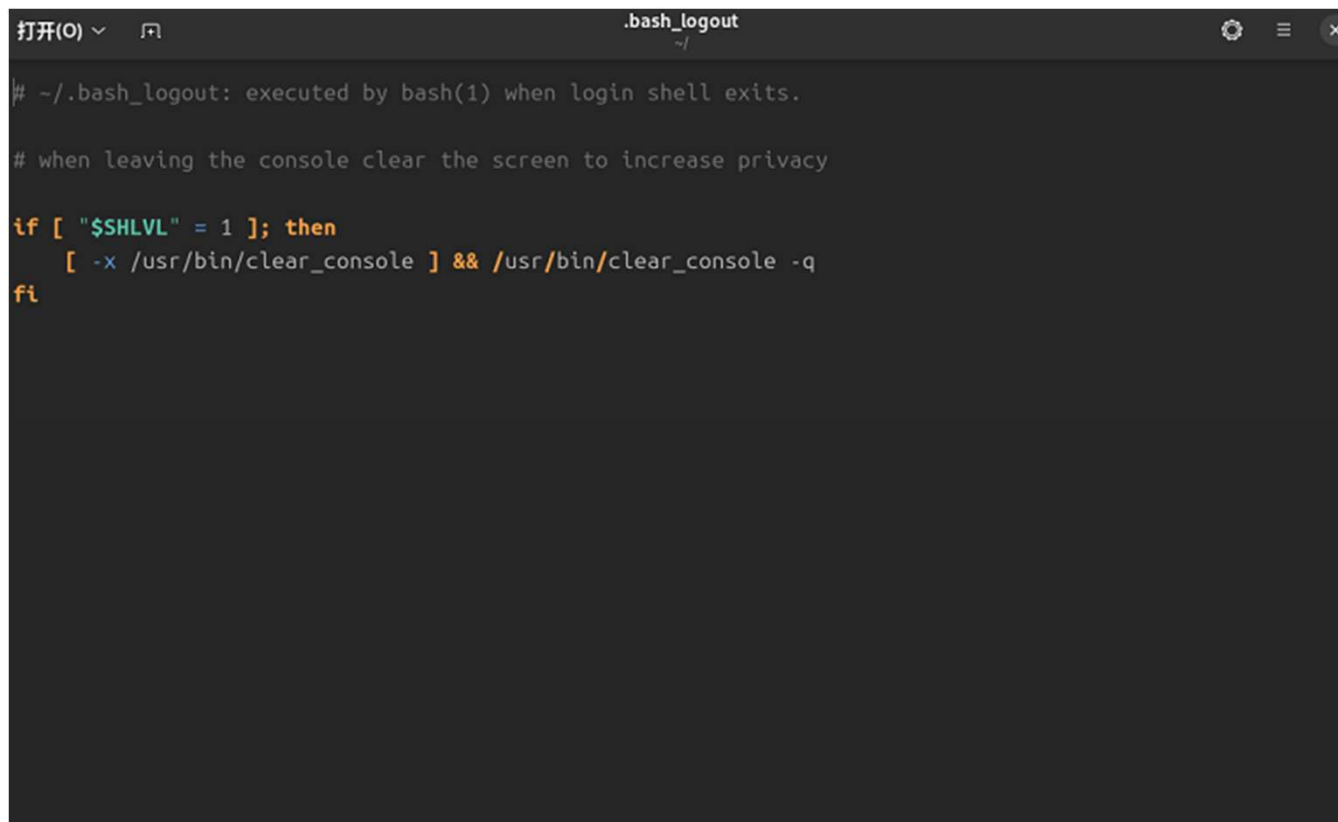
表现形式 – 文件资源管理器



用于访问文件

文件可以选择打开方式
文件可以选择下载到本地

表现形式 – 记事本



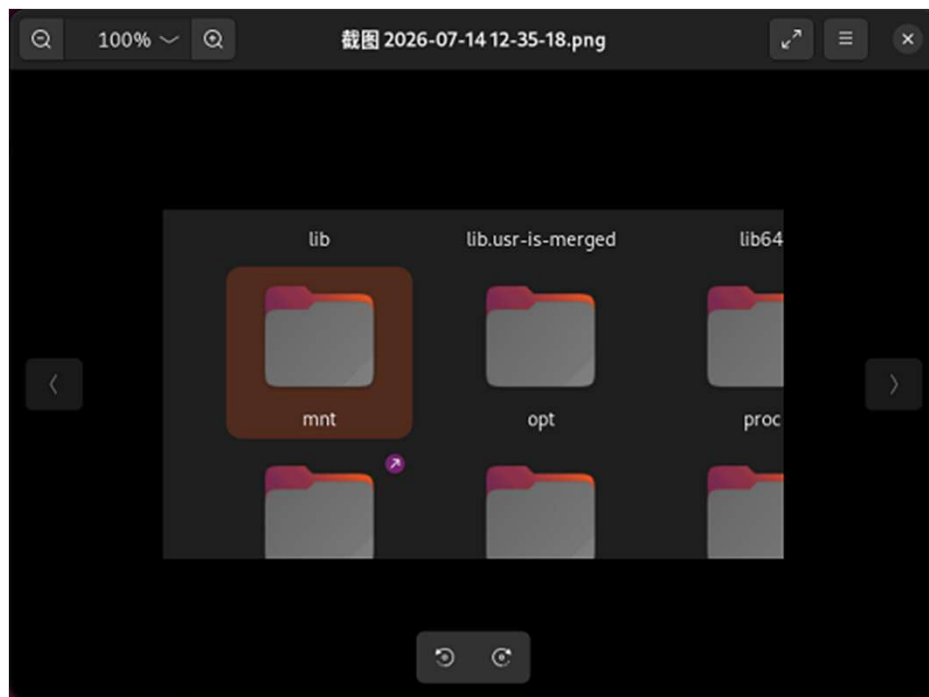
The image shows a terminal window with a dark background. The title bar at the top reads ".bash_logout" and shows standard window controls. The terminal content is as follows:

```
# ~/.bash_logout: executed by bash(1) when login shell exits.  
  
# when leaving the console clear the screen to increase privacy  
  
if [ "$SHLVL" = 1 ]; then  
    [ -x /usr/bin/clear_console ] && /usr/bin/clear_console -q  
fi
```

用于查看文本文件

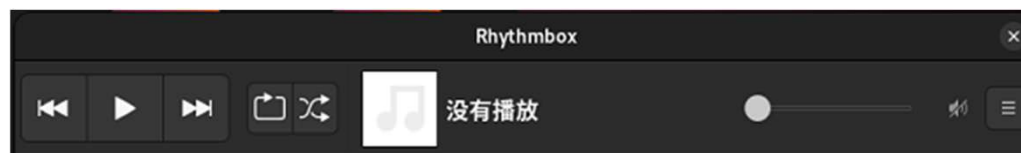
不需要做语法高亮
(愿意做也可以)

表现形式 – 图像查看器



用于查看图像文件

表现形式 – 音频播放器



用于播放音频，做最简单的就可以了

表现形式 – EILnA界面



表现形式 – 交互性谜题

类似这种：[数独在线](#)

当然所有的类型，界面的设计风格要统一。

同时也可以减少提示，比如数独就只有数独的格子，没有选项栏。

关键是以交互性的方式展现谜题，引导、规则等都可以在别的部分完成，所以与解谜游戏还是有所区别的。

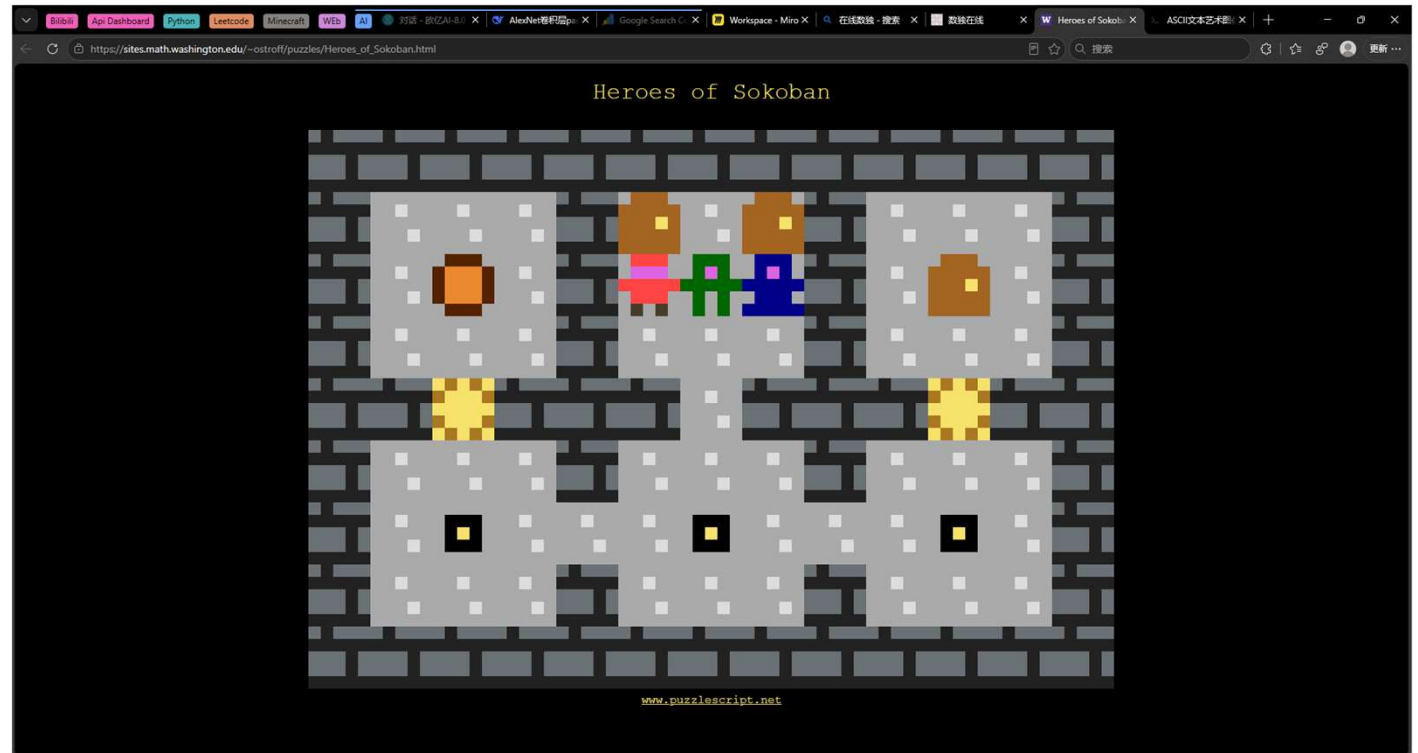


表现形式 – 解谜游戏

@JDK触发器 要求的解谜游戏类型。

可参考 [Heroes of Sokoban](https://sites.math.washington.edu/~ostroff/puzzles/Heroes_of_Sokoban.html)
这个属于纯游戏了。

由JDK触发器负责Idea。



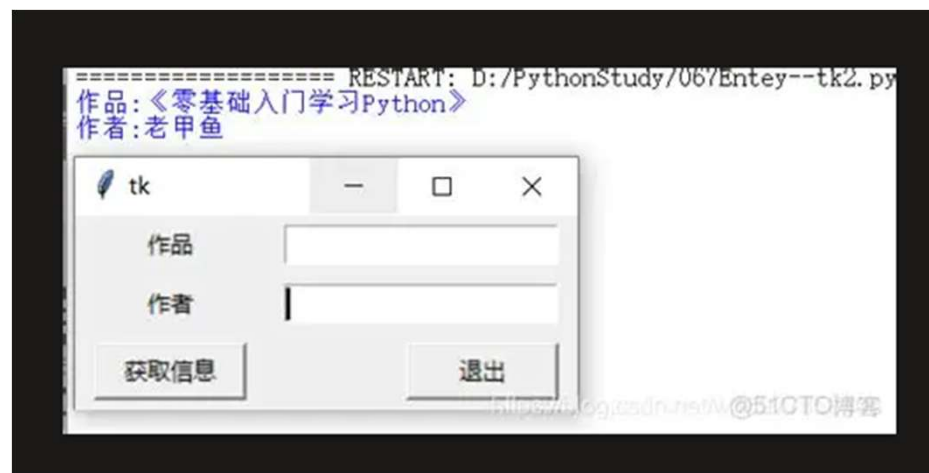
表现形式 – 通用谜题

几乎所有的谜题都需要验证答案。

答案验证的界面可以非常简单。如图所示

当然，这个部分可以和前面的谜题统一为可执行文件的展示。

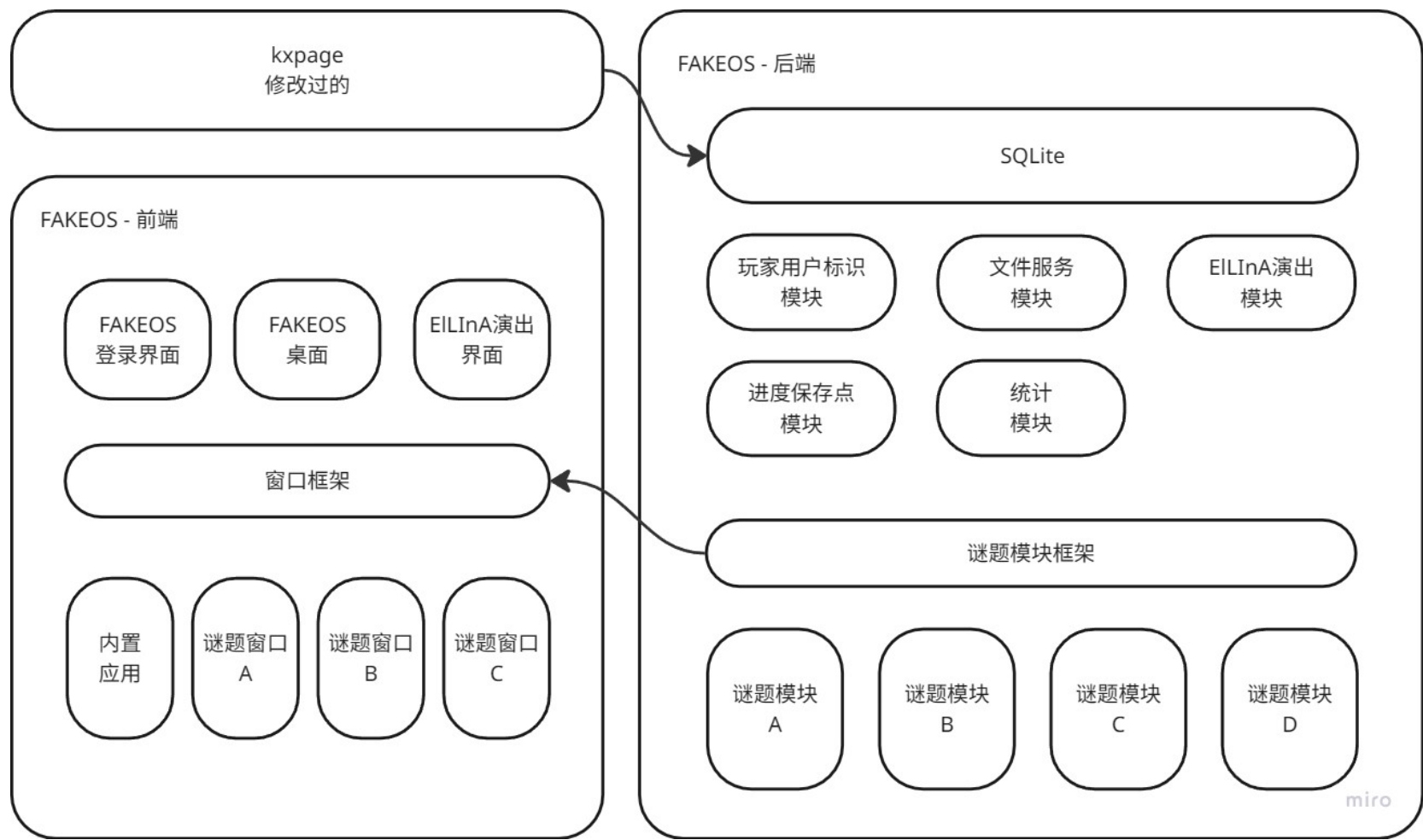
实现细节详见 **技术实现 – 窗口** 部分。



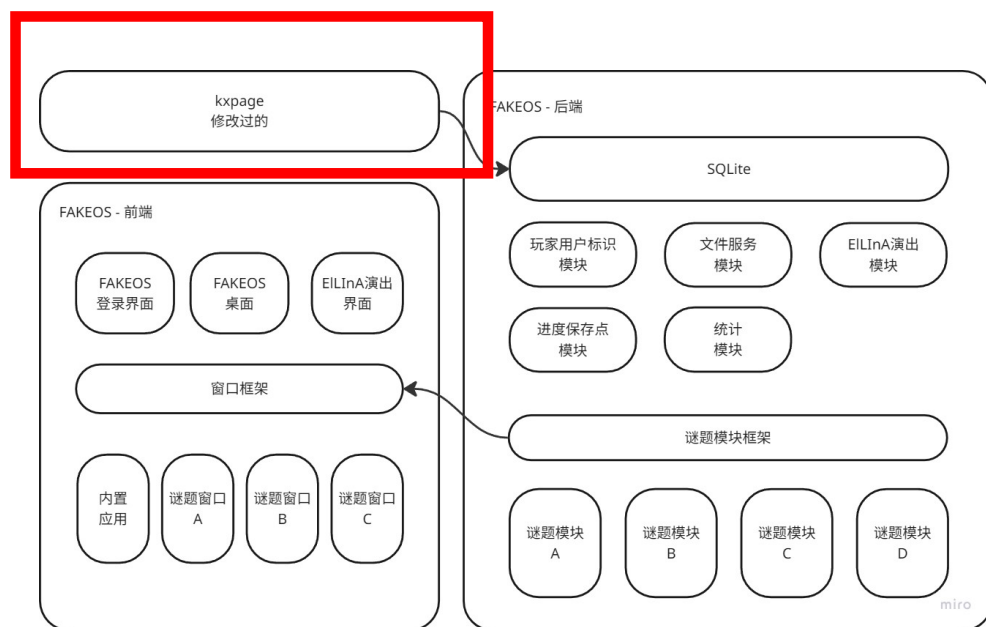
当前的结构还是很复杂的，仅暂时做一个框架的搭建。

对于每个具体的模块，如何设计，数据流如何，细节的实现方案，都待定。
在进行这些模块的编写时，再按具体问题具体分析。

技术实现 – Overview



技术实现 – KXPAGE



Kxpage 用作整个网站的入口。

参考 **剧情-Intro** 部分，用于将玩家引入到FAKEOS内。

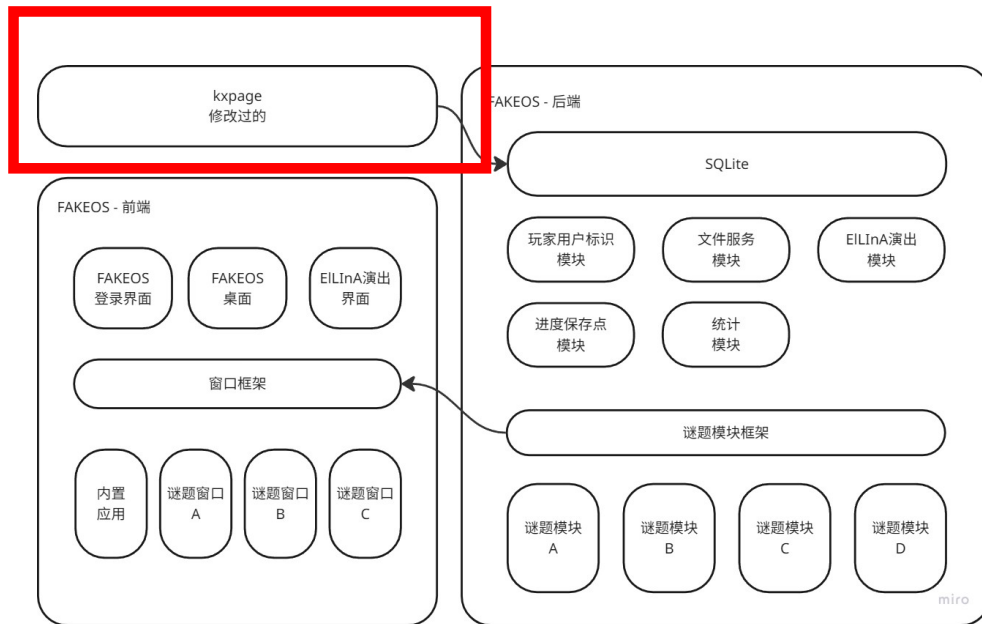
这边主要修改已有的kxpage源码，添加一个入口跳转即可。入口的形式关键看剧情设计，考虑直接在控制台中输出提示即可。

计划所有的网站共用一个nginx，用不同的路由区分。

kxpage: xxx/kxpage

fakeos: xxx/console

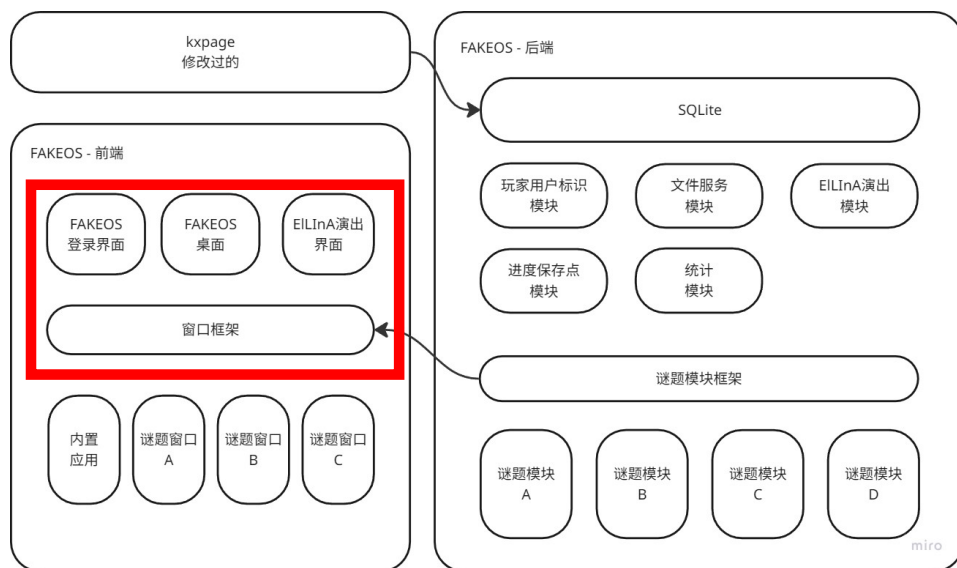
技术实现 – KXPAGE



Kxpage可能还需要对用户做唯一标识。
如果可能的话，可以用LocalStorage自动标识。
同时支持注册账户，重新获取LocalStorage。
如果没有注册，
清空LocalStorage就会丢失所有的进度。

这个部分需要和 FAKEOS – 后端 联动。

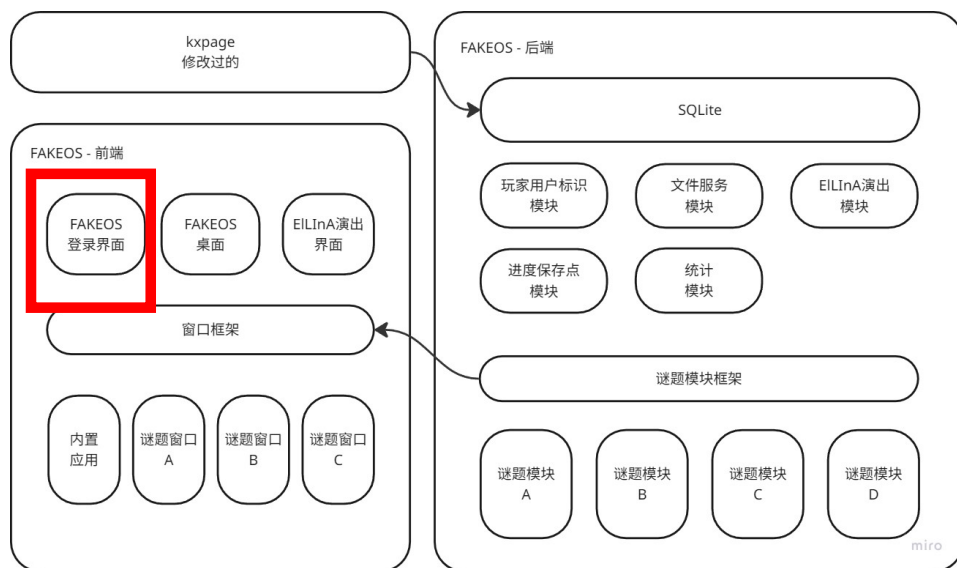
技术实现 – FAKEOS 前端



前端主要分为4个模块：

- 1、登录界面
- 2、桌面界面
- 3、EILInA交互界面
- 4、窗口框架

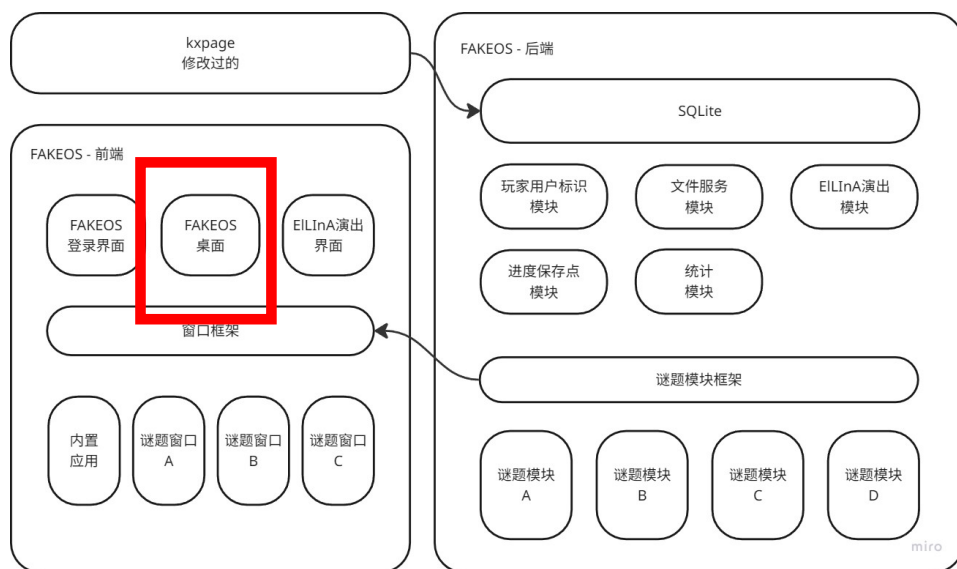
技术实现 – FAKEOS 前端



对于登录界面，就是简单的登录界面。



技术实现 – FAKEOS 前端

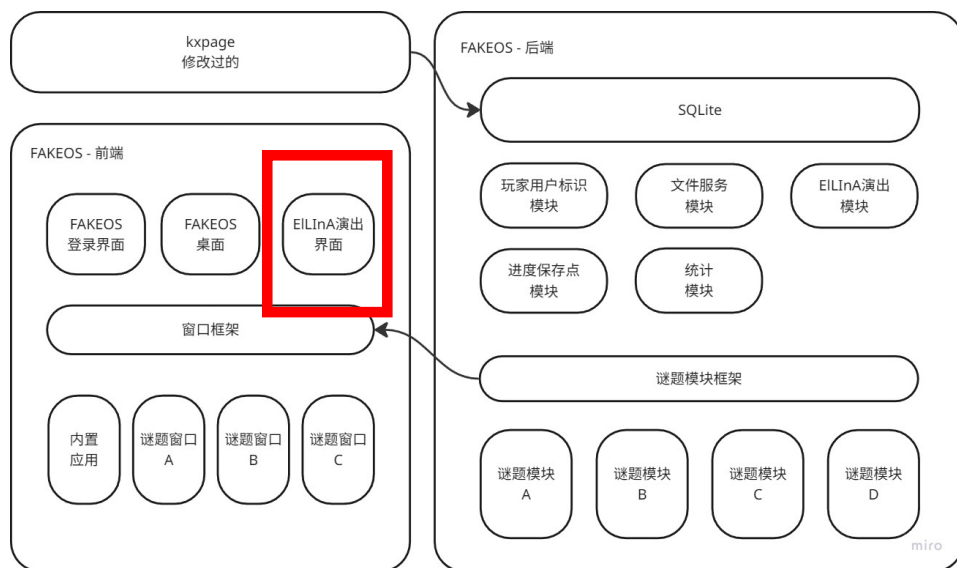


桌面部分，可能不会使用canvas做，比较复杂且无法模块化地加入新的窗口。

- 窗口需要支持拖动、最小化、最大化、关闭。
- 支持状态栏，显示已打开的窗口。
- 支持全部应用的按钮，可以打开全部应用界面。
- 整个桌面环境支持一个消息系统，可以用来控制各个window的状态。

参考 **表现形式 – FakeOS**，以了解需要完成什么样的效果。

技术实现 – FAKEOS 前端



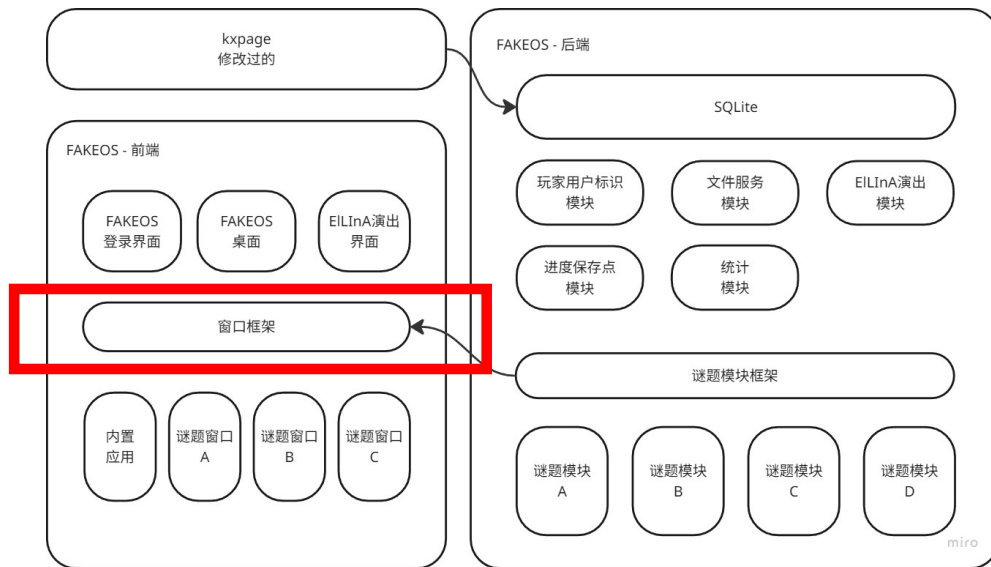
EILInA交互界面是一个比较复杂的界面。该界面仅在PLAYER账户出现。

主体是一个ASCII美术风格的Canvas，需要暴露一些API来提供大字幕/图片展示/抖动/动画的效果。

可能分为两层，一层是ASCII美术的后处理Canvas，一层是用于基础绘图的Canvas。同时需要编写一个基础的导演系统（脚本 -> 演出），用于控制基础绘图Canvas的内容调整。

导演系统需要能够在一定的时候替换原有的状态栏，改为选项来推进剧情，也要支持在一定的时机最小化所有的窗口，用来清空桌面，进行演出。

技术实现 – FAKEOS 前端

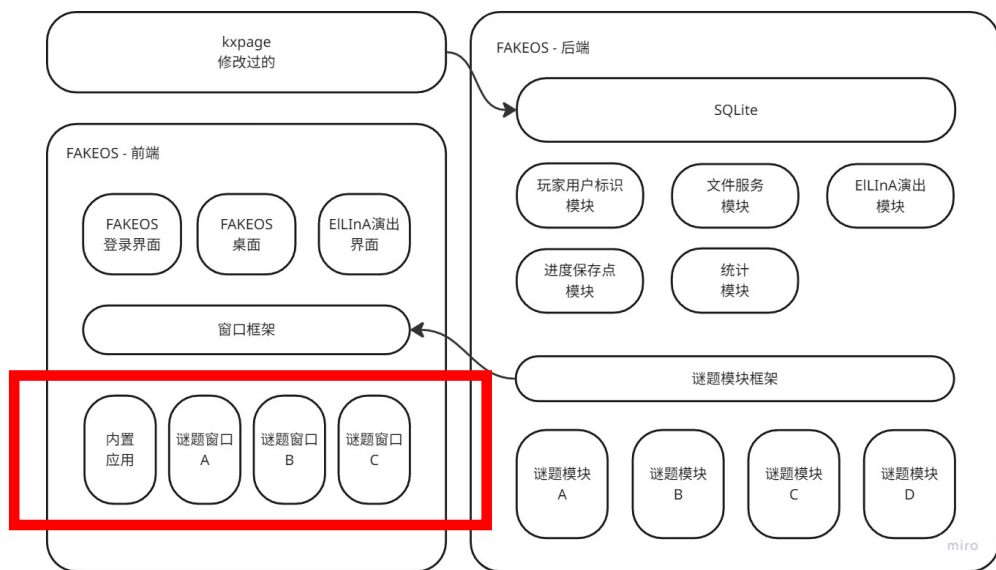


窗口框架可能分为两个部分，一个部分是 WindowService，另一部分是 APIClient。

WindowService 负责暴露窗口的创建和删除，以及窗口之间的消息总线。
APIClient 负责为窗口提供与后端通信的客户端，窗口模块可以通过该客户端方便地使用后端的功能，而不需要自行构建向后端的请求。

对于 APIClient，可能会通过不同谜题模块的唯一 ID 标识来路由某些服务的请求到指定的谜题模块。

技术实现 – FAKEOS 前端



前端除了基本的框架，接下来就是每个窗口的实现。

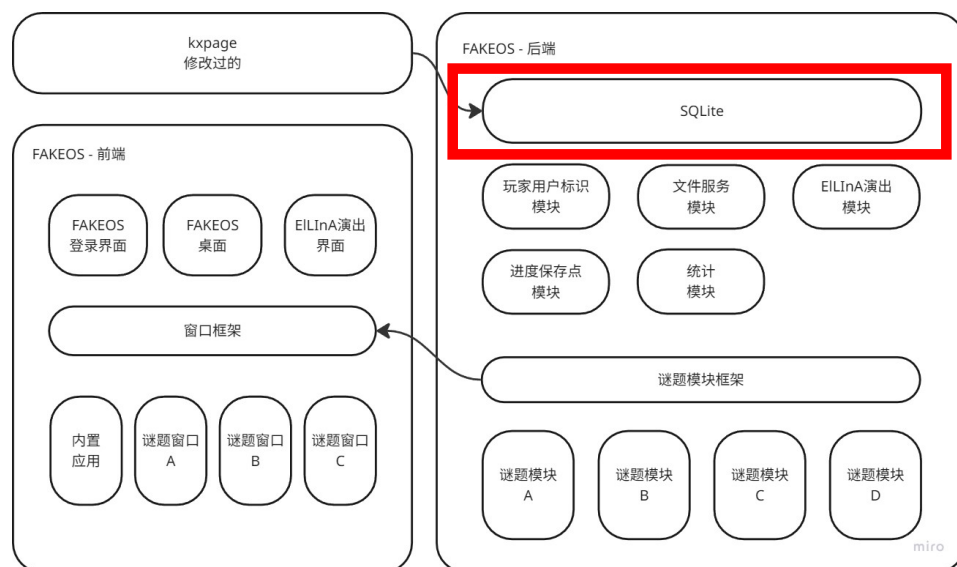
这里考虑模块化的设计，每个窗口都是一个窗口模块的实例。窗口模块懒加载，在开启对应的应用时，会从后端import窗口的JS块。

窗口模块在实现时，只是一个Vue3组件。

内置应用即是内置的窗口模块，比如文件资源管理器、文本查看器、图片查看器。

为了简化开发，谜题模块**不会**提供该谜题所需的窗口模块JS。手动维持窗口模块和谜题模块的数据/接口约定。FAKEOS前端是单个Vue3项目。

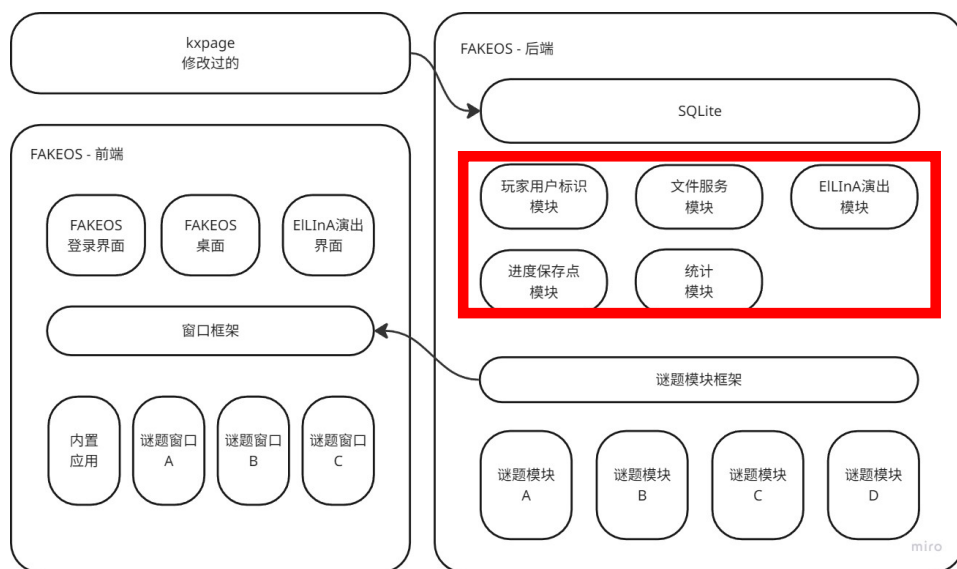
技术实现 – FAKEOS 后端



对于FAKEOS后端，我们考虑使用Python FastAPI构建数据库使用SQLite来方便开发，避免部署大型的SQL数据库。

对于前后端的关系，可以认为全部的玩家数据和谜题数据都是在后端的。前端只是做数据的展示和视觉表现。

技术实现 – FAKEOS 后端



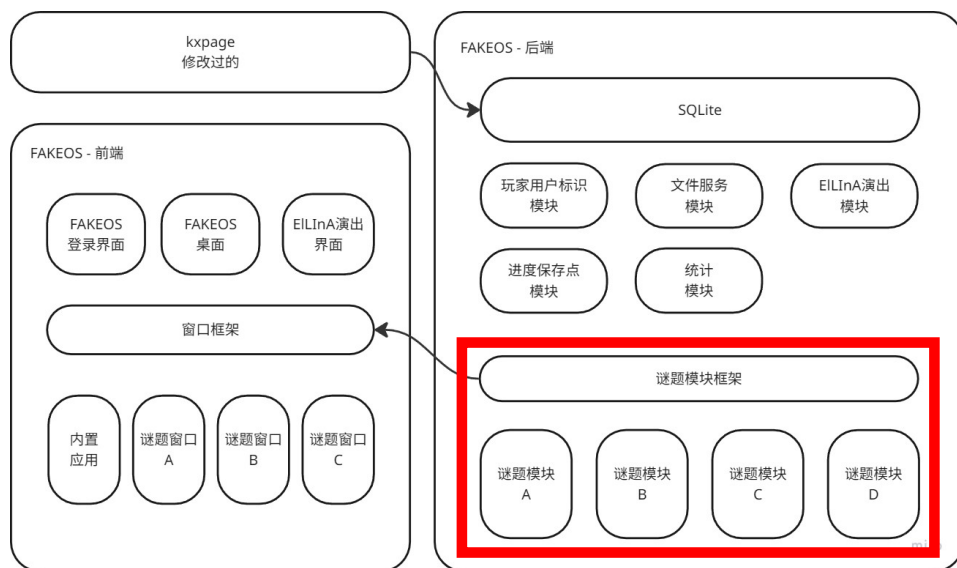
长得像微服务，但是具体实现怎么做待定。

主要有玩家标识、文件服务、进度服务、演出服务、统计服务5个服务。可能还有验证服务。

玩家标识用于区分不同的玩家在服务器中的数据。
文件服务用于提供给玩家虚拟的文件树+文件下载。
进度服务用于管理玩家在整个剧情中的进度。
演出服务用于管理EILInA演出的触发和脚本下载。
统计服务用于管理一些数据的统一收集和提供。

验证服务用于验证玩家提交的谜题答案是否正确。

技术实现 – FAKEOS 后端



对于后端，谜题模块是唯一的数据源，用于提供所有的谜题相关文件、EILInA演出的脚本文件、进度保存点的注册，可统计项的注册，谜题答案验证逻辑的提供。

对于之前提到的6个服务，将从所有的模块收集数据和可注册项，统一提供+路由。

谜题模块框架将提供一些可用的API给谜题模块，用以简化这个信息提供与注册的过程。

为了简化复杂度，考虑直接硬编码的import，而非动态的模块发现与导入。

WHAT'S NEXT

接下来要做的事：

进行简单的人员分组：剧情、谜题、开发、美术。

这个分组只是一个偏向，你偏向于做哪个部分的工作。

WHAT'S NEXT

1、剧情部分

剧情方向的主要任务是将整个大纲落实到可以使用的剧本，同时打磨文本内容和流程逻辑，使其通畅完整合理。于此同时，剧情方向也会涉及部分谜题的IDEA策划。

2、谜题部分

谜题方向主要任务是提供整个解谜的所有谜题设计。对于谜题设计，往往会同时涉及剧情和开发方向的工作：谜题需要在整个剧情大纲中是合理的，同时完成这种谜题的窗口模块（如果不自己编写，也要提供这个窗口的界面和功能描述）。谜题设计需要有巧思。

3、开发部分

开发方向可能是三个部分中最困难的，需要参考整个架构图，然后将每个模块的设计落实，由AI完整编码工作，并且进行单元和联合的测试。如何设计前端的WindowService，如何设计后端的API接口、模块系统，前端的EILInA演出部分Canvas暴露哪些功能，ASCII风格滤镜如何实现。这些都在开发方向的工作范围内。

4、美术部分

美术决定了最后的表现效果，包括但不限于：FAKEOS的界面设计、KXPAGE异常部分的视觉设计、所有类似特效的设计、最后结局立绘的绘制/AI生成、所有谜题窗口的UI设计、EILInA的演出设计、部分谜题需要的图片/视频/音频的设计支持、ASCII滤镜的算法涉及。可能会涉及部分剧情的工作。

WHAT'S NEXT

如果有意愿的话，欢迎在群名片里添加工作方向后缀。

CODING TOOLS

如果进行代码开发的工作，务必使用AI编程工具来辅助，可以使用：

- TRAE：国内版模型免费，需要排队
- Opencode：开源Code CLI，可自由添加提供商
- ClaudeCode：A÷的Code CLI，用CC Switch简化提供商切换。
- Codex：OpenAI，用Codex++使用中转站

CODING TOOLS

模型：

- GPT 5.6/5.5 （中转站/官方）
- Claude （A÷不推荐）
- GLM 5.2 国模T0
- Kimi K2.6 （推理有点慢） K2.7 Code （都是多模态）
- Minimax 至少2.7
- Deepseek V4，上正式版前不推荐，Pro思考链左右脑互博，Flash参数太少。

可以买coding plan，opencode go可以用几乎所有国模，10\$/月，月限额60\$

感谢阅读👁️👁️